

# 140+ BASIC PYTHON PROGRAMS

*This resource can assist you in  
preparing for your interview*

*Piush Kumar Sharma*

## Program 1

Write a Python program to print "Hello Python".

```
In [1]: 1 print("Hello Python")
```

Hello Python

## Program 2

Write a Python program to do arithmetical operations addition and division.

```
In [2]: 1 # Addition
2 num1 = float(input("Enter the first number for addition: "))
3 num2 = float(input("Enter the second number for addition: "))
4 sum_result = num1 + num2
5 print(f"sum: {num1} + {num2} = {sum_result}")
```

Enter the first number for addition: 5  
Enter the second number for addition: 6  
sum: 5.0 + 6.0 = 11.0

```
In [3]: 1 # Division
2 num3 = float(input("Enter the dividend for division: "))
3 num4 = float(input("Enter the divisor for division: "))
4 if num4 == 0:
5     print("Error: Division by zero is not allowed.")
6 else:
7     div_result = num3 / num4
8     print(f"Division: {num3} / {num4} = {div_result}")
```

Enter the dividend for division: 25  
Enter the divisor for division: 5  
Division: 25.0 / 5.0 = 5.0

## Program 3

Write a Python program to find the area of a triangle.

```
In [4]: 1 # Input the base and height from the user
2 base = float(input("Enter the length of the base of the triangle: "))
3 height = float(input("Enter the height of the triangle: "))
4 # Calculate the area of the triangle
5 area = 0.5 * base * height
6 # Display the result
7 print(f"The area of the triangle is: {area}")
```

Enter the length of the base of the triangle: 10  
Enter the height of the triangle: 15  
The area of the triangle is: 75.0

## Program 4

Write a Python program to swap two variables.

```
In [5]: 1 # Input two variables
2 a = input("Enter the value of the first variable (a): ")
3 b = input("Enter the value of the second variable (b): ")
4 # Display the original values
5 print(f"Original values: a = {a}, b = {b}")
6 # Swap the values using a temporary variable
7 temp = a
8 a = b
9 b = temp
10 # Display the swapped values
11 print(f"Swapped values: a = {a}, b = {b}")
```

Enter the value of the first variable (a): 5  
Enter the value of the second variable (b): 9  
Original values: a = 5, b = 9  
Swapped values: a = 9, b = 5

## Program 5

Write a Python program to generate a random number.

```
In [6]: 1 import random
2 print(f"Random number: {random.randint(1, 100)}")
```

Random number: 89

## Program 6

Write a Python program to convert kilometers to miles.

```
In [7]: 1 kilometers = float(input("Enter distance in kilometers: "))
2
3 # Conversion factor: 1 kilometer = 0.621371 miles
4 conversion_factor = 0.621371
5
6 miles = kilometers * conversion_factor
7
8 print(f"{kilometers} kilometers is equal to {miles} miles")
```

Enter distance in kilometers: 100  
100.0 kilometers is equal to 62.137100000000004 miles

## Program 7

Write a Python program to convert Celsius to Fahrenheit.

```
In [8]: 1 celsius = float(input("Enter temperature in Celsius: "))
        2
        3 # Conversion formula: Fahrenheit = (Celsius * 9/5) + 32
        4 fahrenheit = (celsius * 9/5) + 32
        5
        6 print(f"{celsius} degrees Celsius is equal to {fahrenheit} degrees Fahr
```

Enter temperature in Celsius: 37

37.0 degrees Celsius is equal to 98.6 degrees Fahrenheit

## Program 8

Write a Python program to display calendar.

```
In [9]: 1 import calendar
        2
        3 year = int(input("Enter year: "))
        4 month = int(input("Enter month: "))
        5
        6 cal = calendar.month(year, month)
        7 print(cal)
```

Enter year: 2023

Enter month: 11

November 2023

| Mo | Tu | We | Th | Fr | Sa | Su |
|----|----|----|----|----|----|----|
|    |    | 1  | 2  | 3  | 4  | 5  |
| 6  | 7  | 8  | 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 | 17 | 18 | 19 |
| 20 | 21 | 22 | 23 | 24 | 25 | 26 |
| 27 | 28 | 29 | 30 |    |    |    |

## Program 9

Write a Python program to solve quadratic equation.

The standard form of a quadratic equation is:

$$ax^2 + bx + c = 0$$

where

a, b and c are real numbers and

$$a \neq 0$$

The solutions of this quadratic equation is given by:

$$(-b \pm (b^2 - 4ac)^{1/2})/(2a)$$

In [10]:

```
1 import math
2
3 # Input coefficients
4 a = float(input("Enter coefficient a: "))
5 b = float(input("Enter coefficient b: "))
6 c = float(input("Enter coefficient c: "))
7
8 # Calculate the discriminant
9 discriminant = b**2 - 4*a*c
10
11 # Check if the discriminant is positive, negative, or zero
12 if discriminant > 0:
13     # Two real and distinct roots
14     root1 = (-b + math.sqrt(discriminant)) / (2*a)
15     root2 = (-b - math.sqrt(discriminant)) / (2*a)
16     print(f"Root 1: {root1}")
17     print(f"Root 2: {root2}")
18 elif discriminant == 0:
19     # One real root (repeated)
20     root = -b / (2*a)
21     print(f"Root: {root}")
22 else:
23     # Complex roots
24     real_part = -b / (2*a)
25     imaginary_part = math.sqrt(abs(discriminant)) / (2*a)
26     print(f"Root 1: {real_part} + {imaginary_part}i")
27     print(f"Root 2: {real_part} - {imaginary_part}i")
28
```

Enter coefficient a: 1  
Enter coefficient b: 4  
Enter coefficient c: 8  
Root 1: -2.0 + 2.0i  
Root 2: -2.0 - 2.0i

## Program 10

Write a Python program to swap two variables without temp variable.

In [11]:

```
1 a = 5
2 b = 10
3
4 # Swapping without a temporary variable
5 a, b = b, a
6
7
8 print("After swapping:")
9 print("a =", a)
10 print("b =", b)
11
```

After swapping:  
a = 10  
b = 5

## Program 11

Write a Python Program to Check if a Number is Positive, Negative or Zero.

```
In [12]: 1 num = float(input("Enter a number: "))
2 if num > 0:
3     print("Positive number")
4 elif num == 0:
5     print("Zero")
6 else:
7     print("Negative number")
```

Enter a number: 6.4  
Positive number

## Program 12

Write a Python Program to Check if a Number is Odd or Even.

```
In [13]: 1 num = int(input("Enter a number: "))
2
3 if num%2 == 0:
4     print("This is a even number")
5 else:
6     print("This is a odd number")
```

Enter a number: 3  
This is a odd number

## Program 13

Write a Python Program to Check Leap Year.

```
In [14]: 1 year = int(input("Enter a year: "))
2
3 # divided by 100 means century year (ending with 00)
4 # century year divided by 400 is leap year
5 if (year % 400 == 0) and (year % 100 == 0):
6     print("{0} is a leap year".format(year))
7
8 # not divided by 100 means not a century year
9 # year divided by 4 is a leap year
10 elif (year % 4 == 0) and (year % 100 != 0):
11     print("{0} is a leap year".format(year))
12
13 # if not divided by both 400 (century year) and 4 (not century year)
14 # year is not leap year
15 else:
16     print("{0} is not a leap year".format(year))
```

Enter a year: 2024  
2024 is a leap year

## Program 14

**Write a Python Program to Check Prime Number.**

### Prime Numbers:

A prime number is a whole number that cannot be evenly divided by any other number except for 1 and itself. For example, 2, 3, 5, 7, 11, and 13 are prime numbers because they cannot be divided by any other positive integer except for 1 and their own value.

```
In [15]: 1 num = int(input("Enter a number: "))
2
3 # define a flag variable
4 flag = False
5
6 if num == 1:
7     print(f"{num}, is not a prime number")
8 elif num > 1:
9     # check for factors
10    for i in range(2, num):
11        if (num % i) == 0:
12            flag = True    # if factor is found, set flag to True
13            # break out of loop
14            break
15
16    # check if flag is True
17 if flag:
18     print(f"{num}, is not a prime number")
19 else:
20     print(f"{num}, is a prime number")
```

```
Enter a number: 27
27, is not a prime number
```

## Program 15 ¶

**Write a Python Program to Print all Prime Numbers in an Interval of 1-10.**

```
In [20]: 1 # Python program to display all the prime numbers within an interval
2
3 lower = 1
4 upper = 10
5
6 print("Prime numbers between", lower, "and", upper, "are:")
7
8 for num in range(lower, upper + 1):
9     # all prime numbers are greater than 1
10    if num > 1:
11        for i in range(2, num):
12            if (num % i) == 0:
13                break
14        else:
15            print(num)
```

Prime numbers between 1 and 10 are:

2  
3  
5  
7

## Program 16

**Write a Python Program to Find the Factorial of a Number.**

```
In [21]: 1 num = int(input("Enter a number: "))
2 factorial = 1
3 if num < 0:
4     print("Factorial does not exist for negative numbers")
5 elif num == 0:
6     print("Factorial of 0 is 1")
7 else:
8     for i in range(1, num+1):
9         factorial = factorial*i
10    print(f'The factorial of {num} is {factorial}')
```

Enter a number: 4

The factorial of 4 is 24

## Program 17

**Write a Python Program to Display the multiplication Table.**



```
In [22]: 1 num = int(input("Display multiplication table of: "))
          2
          3 for i in range(1, 11):
          4     print(f"{num} X {i} = {num*i}")
```

Display multiplication table of: 19

```
19 X 1 = 19
19 X 2 = 38
19 X 3 = 57
19 X 4 = 76
19 X 5 = 95
19 X 6 = 114
19 X 7 = 133
19 X 8 = 152
19 X 9 = 171
19 X 10 = 190
```

## Program 18

**Write a Python Program to Print the Fibonacci sequence.**

**Fibonacci sequence:**

The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones, typically starting with 0 and 1. So, the sequence begins with 0 and 1, and the next number is obtained by adding the previous two numbers. This pattern continues indefinitely, generating a sequence that looks like this:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, and so on.

Mathematically, the Fibonacci sequence can be defined using the following recurrence relation:

$$F(0) = 0 \quad F(1) = 1 \quad F(n) = F(n-1) + F(n-2) \text{ for } n > 1$$

```

In [23]: 1 nterms = int(input("How many terms? "))
          2
          3 # first two terms
          4 n1, n2 = 0, 1
          5 count = 0
          6
          7 # check if the number of terms is valid
          8 if nterms <= 0:
          9     print("Please enter a positive integer")
         10 # if there is only one term, return n1
         11 elif nterms == 1:
         12     print("Fibonacci sequence upto",nterms,":")
         13     print(n1)
         14 # generate fibonacci sequence
         15 else:
         16     print("Fibonacci sequence:")
         17     while count < nterms:
         18         print(n1)
         19         nth = n1 + n2
         20         # update values
         21         n1 = n2
         22         n2 = nth
         23         count += 1

```

How many terms? 10  
Fibonacci sequence:

0  
1  
1  
2  
3  
5  
8  
13  
21  
34

## Program 19

**Write a Python Program to Check Armstrong Number?**

**Armstrong Number:**

It is a number that is equal to the sum of its own digits, each raised to a power equal to the number of digits in the number.

For example, let's consider the number 153:

- It has three digits (1, 5, and 3).
- If we calculate  $1^3 + 5^3 + 3^3$ , we get  $1 + 125 + 27$ , which is equal to 153.

So, 153 is an Armstrong number because it equals the sum of its digits raised to the power of the number of digits in the number.

Another example is 9474:

- It has four digits (9, 4, 7, and 4).

- If we calculate  $9^4 + 4^4 + 7^4 + 4^4$ , we get  $6561 + 256 + 2401 + 256$ , which is also equal to 9474.

Therefore, 9474 is an Armstrong number as well.

```
In [25]: 1 num = int(input("Enter a number: "))
2
3 # Calculate the number of digits in num
4 num_str = str(num)
5 num_digits = len(num_str)
6
7 # Initialize variables
8 sum_of_powers = 0
9 temp_num = num
10
11 # Calculate the sum of digits raised to the power of num_digits
12
13 while temp_num > 0:
14     digit = temp_num % 10
15     sum_of_powers += digit ** num_digits
16     temp_num //= 10
17
18 # Check if it's an Armstrong number
19 if sum_of_powers == num:
20     print(f"{num} is an Armstrong number.")
21 else:
22     print(f"{num} is not an Armstrong number.")
23
```

Enter a number: 9474

9474 is an Armstrong number.

## Program 20

Write a Python Program to Find Armstrong Number in an Interval.

```

In [26]: 1 # Input the interval from the user
2 lower = int(input("Enter the lower limit of the interval: "))
3 upper = int(input("Enter the upper limit of the interval: "))
4
5
6 for num in range(lower, upper + 1):    # Iterate through the numbers in the interval
7     order = len(str(num))    # Find the number of digits in 'num'
8     temp_num = num
9     sum = 0
10
11     while temp_num > 0:
12         digit = temp_num % 10
13         sum += digit ** order
14         temp_num //= 10
15
16     # Check if 'num' is an Armstrong number
17     if num == sum:
18         print(num)

```

```

Enter the lower limit of the interval: 10
Enter the upper limit of the interval: 1000
153
370
371
407

```

## Program 21

**Write a Python Program to Find the Sum of Natural Numbers.**

**Natural numbers** are a set of positive integers that are used to count and order objects. They are the numbers that typically start from 1 and continue indefinitely, including all the whole numbers greater than 0. In mathematical notation, the set of natural numbers is often denoted as "N" and can be expressed as:

$$N = 1, 2, 3, 4, 5, 6, 7, 8, \dots$$

```

In [27]: 1 limit = int(input("Enter the limit: "))
2
3 # Initialize the sum
4 sum = 0
5
6 # Use a for loop to calculate the sum of natural numbers
7 for i in range(1, limit + 1):
8     sum += i
9
10 # Print the sum
11 print("The sum of natural numbers up to", limit, "is:", sum)

```

```

Enter the limit: 10
The sum of natural numbers up to 10 is: 55

```

## Program 22

**Write a Python Program to Find LCM.**

**Least Common Multiple (LCM):**

LCM, or Least Common Multiple, is the smallest multiple that is exactly divisible by two or more numbers.

Formula:

For two numbers a and b, the LCM can be found using the formula:

$$\text{LCM}(a, b) = \frac{|a \cdot b|}{\text{GCD}(a, b)}$$

For more than two numbers, you can find the LCM step by step, taking the LCM of pairs of numbers at a time until you reach the last pair.

Note: GCD stands for Greatest Common Divisor.

```
In [1]: 1 # Python Program to find the L.C.M. of two input number
        2
        3 def compute_lcm(x, y):
        4     if x > y:                               # choose the greater number
        5         greater = x
        6     else:
        7         greater = y
        8     while(True):
        9         if((greater % x == 0) and (greater % y == 0)):
        10             lcm = greater
        11             break
        12         greater += 1
        13     return lcm
        14
        15 num1 = int(input('Enter the number: '))
        16 num2 = int(input('Enter the number: '))
        17
        18 print("The L.C.M. is", compute_lcm(num1, num2))
```

Enter the number: 54

Enter the number: 24

The L.C.M. is 216

## Program 23

**Write a Python Program to Find HCF.**

**Highest Common Factor(HCF):**

HCF, or Highest Common Factor, is the largest positive integer that divides two or more numbers without leaving a remainder.

Formula:

For two numbers a and b, the HCF can be found using the formula:

$$\text{HCF}(a, b) = \text{GCD}(a, b)$$

For more than two numbers, you can find the HCF by taking the GCD of pairs of numbers at a time until you reach the last pair.

Note: GCD stands for Greatest Common Divisor.

```
In [2]: 1 # Python program to find H.C.F of two numbers
        2
        3 # define a function
        4 def compute_hcf(x, y):
        5
        6 # choose the smaller number
        7     if x > y:
        8         smaller = y
        9     else:
        10        smaller = x
        11        for i in range(1, smaller+1):
        12            if((x % i == 0) and (y % i == 0)):
        13                hcf = i
        14        return hcf
        15
        16 num1 = int(input('Enter the number: '))
        17 num2 = int(input('Enter the number: '))
        18
        19 print("The H.C.F. is", compute_hcf(num1, num2))
```

Enter the number: 54

Enter the number: 24

The H.C.F. is 6

## Program 24

**Write a Python Program to Convert Decimal to Binary, Octal and Hexadecimal.**

**How can we manually convert a decimal number to binary, octal and hexadecimal?**

Converting a decimal number to binary, octal, and hexadecimal involves dividing the decimal number by the base repeatedly and noting the remainders at each step. Here's a simple example:

Let's convert the decimal number 27 to binary, octal, and hexadecimal.

### 1. Binary:

- Divide 27 by 2. Quotient is 13, remainder is 1. Note the remainder.
- Divide 13 by 2. Quotient is 6, remainder is 1. Note the remainder.
- Divide 6 by 2. Quotient is 3, remainder is 0. Note the remainder.
- Divide 3 by 2. Quotient is 1, remainder is 1. Note the remainder.
- Divide 1 by 2. Quotient is 0, remainder is 1. Note the remainder.

Reading the remainders from bottom to top, the binary representation of 27 is 11011.

### 2. Octal:

- Divide 27 by 8. Quotient is 3, remainder is 3. Note the remainder.
- Divide 3 by 8. Quotient is 0, remainder is 3. Note the remainder.

Reading the remainders from bottom to top, the octal representation of 27 is 33.

### 3. Hexadecimal:

- Divide 27 by 16. Quotient is 1, remainder is 11 (B in hexadecimal). Note the remainder.

Reading the remainders, the hexadecimal representation of 27 is 1B.

So, in summary:

- Binary: 27 in decimal is 11011 in binary.
- Octal: 27 in decimal is 33 in octal.
- Hexadecimal: 27 in decimal is 1B in hexadecimal.

In [3]:

```
1 dec_num = int(input('Enter a decimal number: '))
2
3 print("The decimal value of", dec_num, "is:")
4 print(bin(dec_num), "in binary.")
5 print(oct(dec_num), "in octal.")
6 print(hex(dec_num), "in hexadecimal.")
```

```
Enter a decimal number: 27
The decimal value of 27 is:
0b11011 in binary.
0o33 in octal.
0x1b in hexadecimal.
```

## Program 25

**Write a Python Program To Find ASCII value of a character.**

**ASCII value:**

ASCII, or American Standard Code for Information Interchange, is a character encoding standard that uses numeric values to represent characters. Each ASCII character is assigned a unique 7-bit or 8-bit binary number, allowing computers to exchange information and display text in a consistent way. The ASCII values range from 0 to 127 (for 7-bit ASCII) or 0 to 255 (for 8-bit ASCII), with each value corresponding to a specific character, such as letters, digits, punctuation marks, and control characters.

In [4]:

```
1 char = str(input("Enter the character: "))
2 print("The ASCII value of '" + char + "' is", ord(char))
```

```
Enter the character: P
The ASCII value of 'P' is 80
```

## Program 26

**Write a Python Program to Make a Simple Calculator with 4 basic mathematical operations.**

In [5]:

```
1  # This function adds two numbers
2  def add(x, y):
3      return x + y
4
5  # This function subtracts two numbers
6  def subtract(x, y):
7      return x - y
8
9  # This function multiplies two numbers
10 def multiply(x, y):
11     return x * y
12
13 # This function divides two numbers
14 def divide(x, y):
15     return x / y
16
17
18 print("Select operation.")
19 print("1.Add")
20 print("2.Subtract")
21 print("3.Multiply")
22 print("4.Divide")
23
24 while True:
25     # take input from the user
26     choice = input("Enter choice(1/2/3/4): ")
27
28     # check if choice is one of the four options
29     if choice in ('1', '2', '3', '4'):
30         try:
31             num1 = float(input("Enter first number: "))
32             num2 = float(input("Enter second number: "))
33         except ValueError:
34             print("Invalid input. Please enter a number.")
35             continue
36
37         if choice == '1':
38             print(num1, "+", num2, "=", add(num1, num2))
39
40         elif choice == '2':
41             print(num1, "-", num2, "=", subtract(num1, num2))
42
43         elif choice == '3':
44             print(num1, "*", num2, "=", multiply(num1, num2))
45
46         elif choice == '4':
47             print(num1, "/", num2, "=", divide(num1, num2))
48
49     # check if user wants another calculation
50     # break the while loop if answer is no
51     next_calculation = input("Let's do next calculation? (yes/no): ")
52     if next_calculation == "no":
53         break
54 else:
55     print("Invalid Input")
```



```
Select operation.  
1.Add  
2.Subtract  
3.Multiply  
4.Divide  
Enter choice(1/2/3/4): 1  
Enter first number: 5  
Enter second number: 6  
5.0 + 6.0 = 11.0  
Let's do next calculation? (yes/no): yes  
Enter choice(1/2/3/4): 2  
Enter first number: 50  
Enter second number: 5  
50.0 - 5.0 = 45.0  
Let's do next calculation? (yes/no): yes  
Enter choice(1/2/3/4): 3  
Enter first number: 22  
Enter second number: 2  
22.0 * 2.0 = 44.0  
Let's do next calculation? (yes/no): yes  
Enter choice(1/2/3/4): 4  
Enter first number: 99  
Enter second number: 9  
99.0 / 9.0 = 11.0  
Let's do next calculation? (yes/no): no
```

## Program 27

**Write a Python Program to Display Fibonacci Sequence Using Recursion.**

### **Fibonacci sequence:**

The Fibonacci sequence is a series of numbers in which each number is the sum of the two preceding ones, usually starting with 0 and 1. In mathematical terms, it is defined by the recurrence relation ( $F(n) = F(n-1) + F(n-2)$ ), with initial conditions ( $F(0) = 0$ ) and ( $F(1) = 1$ ). The sequence begins: 0, 1, 1, 2, 3, 5, 8, 13, 21, and so on. The Fibonacci sequence has widespread applications in mathematics, computer science, nature, and art.

```

In [9]: 1 # Python program to display the Fibonacci sequence
        2
        3 def recur_fibo(n):
        4     if n <= 1:
        5         return n
        6     else:
        7         return(recur_fibo(n-1) + recur_fibo(n-2))
        8
        9 nterms = int(input("Enter the number of terms (greater than 0): "))
       10
       11 # check if the number of terms is valid
       12 if nterms <= 0:
       13     print("Plese enter a positive integer")
       14 else:
       15     print("Fibonacci sequence:")
       16     for i in range(nterms):
       17         print(recur_fibo(i))

```

Enter the number of terms (greater than 0): 8  
Fibonacci sequence:

0  
1  
1  
2  
3  
5  
8  
13

## Program 28

**Write a Python Program to Find Factorial of Number Using Recursion.**

The factorial of a non-negative integer (  $n$  ) is the product of all positive integers less than or equal to (  $n$  ). It is denoted by (  $n!$  ) and is defined as:

$$n! = n \times (n - 1) \times (n - 2) \times \dots \times 3 \times 2 \times 1$$

For example:

- $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$
- $0!$  is defined to be 1.

Factorials are commonly used in mathematics, especially in combinatorics and probability, to count the number of ways a set of elements can be arranged or selected.

```
In [11]: 1 # Factorial of a number using recursion
2
3 def recur_factorial(n):
4     if n == 1:
5         return n
6     else:
7         return n*recur_factorial(n-1)
8
9 num = int(input("Enter the number: "))
10
11 # check if the number is negative
12 if num < 0:
13     print("Sorry, factorial does not exist for negative numbers")
14 elif num == 0:
15     print("The factorial of 0 is 1")
16 else:
17     print("The factorial of", num, "is", recur_factorial(num))
```

Enter the number: 7

The factorial of 7 is 5040

## Program 29

**Write a Python Program to calculate your Body Mass Index.**

**Body Mass Index (BMI)** is a measure of body fat based on an individual's weight and height. It is commonly used as a screening tool to categorize individuals into different weight status categories, such as underweight, normal weight, overweight, and obesity.

The BMI is calculated using the following formula:

$$\text{BMI} = \frac{\text{Weight (kg)}}{\text{Height (m)}^2}$$

Alternatively, in the imperial system:

$$\text{BMI} = \frac{\text{Weight (lb)}}{\text{Height (in)}^2} \times 703$$

BMI provides a general indication of body fatness but does not directly measure body fat or distribution. It is widely used in public health and clinical settings as a quick and simple tool to assess potential health risks associated with weight. Different BMI ranges are associated with different health categories, but it's important to note that BMI has limitations and does not account for factors such as muscle mass or distribution of fat.

```
In [12]: 1 def bodymassindex(height, weight):
2         return round((weight / height**2),2)
3
4
5 h = float(input("Enter your height in meters: "))
6 w = float(input("Enter your weight in kg: "))
7
8
9 print("Welcome to the BMI calculator.")
10
11 bmi = bodymassindex(h, w)
12 print("Your BMI is: ", bmi)
13
14
15 if bmi <= 18.5:
16     print("You are underweight.")
17 elif 18.5 < bmi <= 24.9:
18     print("Your weight is normal.")
19 elif 25 < bmi <= 29.29:
20     print("You are overweight.")
21 else:
22     print("You are obese.")
```

```
Enter your height in meters: 1.8
Enter your weight in kg: 70
Welcome to the BMI calculator.
Your BMI is: 21.6
Your weight is normal.
```

## Program 30

**Write a Python Program to calculate the natural logarithm of any number.**

The **natural logarithm**, often denoted as  $\ln$ , is a mathematical function that represents the logarithm to the base  $e$ , where  $e$  is the mathematical constant approximately equal to 2.71828. In other words, for a positive number  $x$ , the natural logarithm of  $x$  is the exponent  $y$  that satisfies the equation  $e^y = x$ .

Mathematically, the natural logarithm is expressed as:

$$\ln(x)$$

It is commonly used in various branches of mathematics, especially in calculus and mathematical analysis, as well as in fields such as physics, economics, and engineering. The natural logarithm has properties that make it particularly useful in situations involving exponential growth or decay.

```
In [13]: 1 import math
2
3 num = float(input("Enter a number: "))
4
5 if num <= 0:
6     print("Please enter a positive number.")
7 else:
8     # Calculate the natural logarithm (base e) of the number
9     result = math.log(num)
10    print(f"The natural logarithm of {num} is: {result}")
```

Enter a number: 1.4

The natural logarithm of 1.4 is: 0.3364722366212129

## Program 31

Write a Python Program for cube sum of first n natural numbers?

```
In [14]: 1 def cube_sum_of_natural_numbers(n):
2         if n <= 0:
3             return 0
4         else:
5             total = sum([i**3 for i in range(1, n + 1)])
6             return total
7
8 # Input the number of natural numbers
9 n = int(input("Enter the value of n: "))
10
11 if n <= 0:
12     print("Please enter a positive integer.")
13 else:
14     result = cube_sum_of_natural_numbers(n)
15     print(f"The cube sum of the first {n} natural numbers is: {result}")
```

Enter the value of n: 7

The cube sum of the first 7 natural numbers is: 784

## Program 32

Write a Python Program to find sum of array.

In Python, an **array** is a data structure used to store a collection of elements, each identified by an index or a key. Unlike some other programming languages, Python does not have a built-in array type. Instead, the most commonly used array-like data structure is the list.

A list in Python is a dynamic array, meaning it can change in size during runtime. Elements in a list can be of different data types, and you can perform various operations such as adding, removing, or modifying elements. Lists are defined using square brackets `[]` and can be indexed and sliced to access specific elements or sublists.

Example of a simple list in Python:

```
my_list = [1, 2, 3, 4, 5]
```

This list can be accessed and manipulated using various built-in functions and methods in Python.

```
In [15]: 1 # Finding Sum of Array Using sum()
2 arr = [1,2,3]
3
4 ans = sum(arr)
5
6 print('Sum of the array is ', ans)
```

Sum of the array is 6

```
In [16]: 1 # Function to find the sum of elements in an array
2 def sum_of_array(arr):
3     total = 0 # Initialize a variable to store the sum
4
5     for element in arr:
6         total += element # Add each element to the total
7
8     return total
9
10 # Example usage:
11 array = [1, 2, 3]
12 result = sum_of_array(array)
13 print("Sum of the array:", result)
```

Sum of the array: 6

## Program 33

Write a Python Program to find largest element in an array.

```
In [18]: 1 def find_largest_element(arr):
2     if not arr:
3         return "Array is empty"
4
5     # Initialize the first element as the largest
6     largest_element = arr[0]
7
8     # Iterate through the array to find the largest element
9     for element in arr:
10        if element > largest_element:
11            largest_element = element
12
13    return largest_element
14
15 # Example usage:
16 my_array = [10, 20, 30, 99]
17 result = find_largest_element(my_array)
18 print(f"The largest element in the array is: {result}")
19
```

The largest element in the array is: 99

## Program 34

Write a Python Program for array rotation.

```
In [19]: 1 def rotate_array(arr, d):
2         n = len(arr)
3
4         # Check if 'd' is valid, it should be within the range of array length
5         if d < 0 or d >= n:
6             return "Invalid rotation value"
7
8         # Create a new array to store the rotated elements.
9         rotated_arr = [0] * n
10
11        # Perform the rotation.
12        for i in range(n):
13            rotated_arr[i] = arr[(i + d) % n]
14
15        return rotated_arr
16
17        # Input array
18        arr = [1, 2, 3, 4, 5]
19
20        # Number of positions to rotate
21        d = 2
22
23        # Call the rotate_array function
24        result = rotate_array(arr, d)
25
26        # Print the rotated array
27        print("Original Array:", arr)
28        print("Rotated Array:", result)
29
```

Original Array: [1, 2, 3, 4, 5]

Rotated Array: [3, 4, 5, 1, 2]

## Program 35

Write a Python Program to Split the array and add the first part to the end?

```
In [20]: 1 def split_and_add(arr, k):
2         if k <= 0 or k >= len(arr):
3             return arr
4
5         # Split the array into two parts
6         first_part = arr[:k]
7         second_part = arr[k:]
8
9         # Add the first part to the end of the second part
10        result = second_part + first_part
11
12        return result
13
14    # Test the function
15    arr = [1, 2, 3, 4, 5]
16    k = 3
17    result = split_and_add(arr, k)
18    print("Original Array:", arr)
19    print("Array after splitting and adding:", result)
```

Original Array: [1, 2, 3, 4, 5]

Array after splitting and adding: [4, 5, 1, 2, 3]

## Program 36

**Write a Python Program to check if given array is Monotonic.**

- A monotonic array is one that is entirely non-increasing or non-decreasing.

```
In [21]: 1 def is_monotonic(arr):
2         increasing = decreasing = True
3
4         for i in range(1, len(arr)):
5             if arr[i] > arr[i - 1]:
6                 decreasing = False
7             elif arr[i] < arr[i - 1]:
8                 increasing = False
9
10        return increasing or decreasing
11
12    # Test the function
13    arr1 = [1, 2, 2, 3] # Monotonic (non-decreasing)
14    arr2 = [3, 2, 1]    # Monotonic (non-increasing)
15    arr3 = [1, 3, 2, 4] # Not monotonic
16
17    print("arr1 is monotonic:", is_monotonic(arr1))
18    print("arr2 is monotonic:", is_monotonic(arr2))
19    print("arr3 is monotonic:", is_monotonic(arr3))
```

arr1 is monotonic: True

arr2 is monotonic: True

arr3 is monotonic: False



## Program 37

Write a Python Program to Add Two Matrices.

```
In [1]: 1 # Function to add two matrices
2 def add_matrices(mat1, mat2):
3     # Check if the matrices have the same dimensions
4     if len(mat1) != len(mat2) or len(mat1[0]) != len(mat2[0]):
5         return "Matrices must have the same dimensions for addition"
6
7     # Initialize an empty result matrix with the same dimensions
8     result = []
9     for i in range(len(mat1)):
10         row = []
11         for j in range(len(mat1[0])):
12             row.append(mat1[i][j] + mat2[i][j])
13         result.append(row)
14
15     return result
16
17 # Input matrices
18 matrix1 = [
19     [1, 2, 3],
20     [4, 5, 6],
21     [7, 8, 9]
22 ]
23
24 matrix2 = [
25     [9, 8, 7],
26     [6, 5, 4],
27     [3, 2, 1]
28 ]
29
30 # Call the add_matrices function
31 result_matrix = add_matrices(matrix1, matrix2)
32
33 # Display the result
34 if isinstance(result_matrix, str):
35     print(result_matrix)
36 else:
37     print("Sum of matrices:")
38     for row in result_matrix:
39         print(row)
40
```

```
Sum of matrices:
[10, 10, 10]
[10, 10, 10]
[10, 10, 10]
```

## Program 38

Write a Python Program to Multiply Two Matrices.

```

In [2]: 1 # Function to multiply two matrices
2 def multiply_matrices(mat1, mat2):
3     # Determine the dimensions of the input matrices
4     rows1 = len(mat1)
5     cols1 = len(mat1[0])
6     rows2 = len(mat2)
7     cols2 = len(mat2[0])
8
9     # Check if multiplication is possible
10    if cols1 != rows2:
11        return "Matrix multiplication is not possible. Number of columns of first matrix is not equal to number of rows of second matrix."
12
13    # Initialize the result matrix with zeros
14    result = [[0 for _ in range(cols2)] for _ in range(rows1)]
15
16    # Perform matrix multiplication
17    for i in range(rows1):
18        for j in range(cols2):
19            for k in range(cols1):
20                result[i][j] += mat1[i][k] * mat2[k][j]
21
22    return result
23
24    # Example matrices
25    matrix1 = [[1, 2, 3],
26               [4, 5, 6]]
27
28    matrix2 = [[7, 8],
29               [9, 10],
30               [11, 12]]
31
32    # Multiply the matrices
33    result_matrix = multiply_matrices(matrix1, matrix2)
34
35    # Display the result
36    if isinstance(result_matrix, str):
37        print(result_matrix)
38    else:
39        print("Result of matrix multiplication:")
40        for row in result_matrix:
41            print(row)

```

```

Result of matrix multiplication:
[58, 64]
[139, 154]

```

## Program 39

Write a Python Program to Transpose a Matrix.

In [3]:

```
1  # Function to transpose a matrix
2  def transpose_matrix(matrix):
3      rows, cols = len(matrix), len(matrix[0])
4      # Create an empty matrix to store the transposed data
5      result = [[0 for _ in range(rows)] for _ in range(cols)]
6
7      for i in range(rows):
8          for j in range(cols):
9              result[j][i] = matrix[i][j]
10
11     return result
12
13 # Input matrix
14 matrix = [
15     [1, 2, 3],
16     [4, 5, 6]
17 ]
18
19 # Transpose the matrix
20 transposed_matrix = transpose_matrix(matrix)
21
22 # Print the transposed matrix
23 for row in transposed_matrix:
24     print(row)
25
```

```
[1, 4]
[2, 5]
[3, 6]
```

## Program 40

Write a Python Program to Sort Words in Alphabetic Order.

```
In [4]: 1 # Program to sort alphabetically the words form a string provided by th
2
3 my_str = input("Enter a string: ")
4
5 # breakdown the string into a list of words
6 words = [word.capitalize() for word in my_str.split()]
7
8 # sort the list
9 words.sort()
10
11 # display the sorted words
12
13 print("The sorted words are:")
14 for word in words:
15     print(word)
```

Enter a string: suresh ramesh vibhuti gulgule raji ram shyam ajay  
The sorted words are:

Ajay  
Gulgule  
Raji  
Ram  
Ramesh  
Shyam  
Suresh  
Vibhuti

## Program 41

**Write a Python Program to Remove Punctuation From a String.**

```
In [5]: 1 # define punctuation
2 punctuations = '!()-[]{};:","<>./?@$%^&*~_''
3
4
5 # To take input from the user
6 my_str = input("Enter a string: ")
7
8 # remove punctuation from the string
9 no_punct = ""
10 for char in my_str:
11     if char not in punctuations:
12         no_punct = no_punct + char
13
14 # display the unpunctuated string
15 print(no_punct)
```

Enter a string: Hello!!!, he said ---and went  
Hello he said and went

## Program 42

In [ ]:

1

In [ ]:

1

## Program 43

Write a Python program to check if the given number is a Disarium Number.

A **Disarium number** is a number that is equal to the sum of its digits each raised to the power of its respective position. For example, 89 is a Disarium number because  $8^1 + 9^2 = 8 + 81 = 89$ .

In [1]:

```
1 def is_disarium(number):
2     # Convert the number to a string to iterate over its digits
3     num_str = str(number)
4
5     # Calculate the sum of digits raised to their respective positions
6     digit_sum = sum(int(i) ** (index + 1) for index, i in enumerate(num_str))
7
8     # Check if the sum is equal to the original number
9     return digit_sum == number
10
11 # Input a number from the user
12 try:
13     num = int(input("Enter a number: "))
14
15     # Check if it's a Disarium number
16     if is_disarium(num):
17         print(f"{num} is a Disarium number.")
18     else:
19         print(f"{num} is not a Disarium number.")
20 except ValueError:
21     print("Invalid input. Please enter a valid number.")
```

Enter a number: 89

89 is a Disarium number.

## Program 44

Write a Python program to print all disarium numbers between 1 to 100.

```
In [2]: 1 def is_disarium(num):
2         num_str = str(num)
3         digit_sum = sum(int(i) ** (index + 1) for index, i in enumerate(num_str))
4         return num == digit_sum
5
6 disarium_numbers = [num for num in range(1, 101) if is_disarium(num)]
7
8 print("Disarium numbers between 1 and 100:")
9 for num in disarium_numbers:
10     print(num, end=" | ")
```

Disarium numbers between 1 and 100:

1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 89 |

## Program 45

**Write a Python program to check if the given number is Happy Number.**

**Happy Number:** A Happy Number is a positive integer that, when you repeatedly replace the number by the sum of the squares of its digits and continue the process, eventually reaches 1. If the process never reaches 1 but instead loops endlessly in a cycle, the number is not a Happy Number.

For example:

19 is a Happy Number because:

$$1^2 + 9^2 = 82$$

$$8^2 + 2^2 = 68$$

$$6^2 + 8^2 = 100$$

$$1^2 + 0^2 + 0^2 = 1$$

The process reaches 1, so 19 is a Happy Number.

```
In [3]: 1 def is_happy_number(num):
2         seen = set() # To store previously seen numbers
3
4         while num != 1 and num not in seen:
5             seen.add(num)
6             num = sum(int(i) ** 2 for i in str(num))
7
8         return num == 1
9
10 # Test the function with a number
11 num = int(input("Enter a number: "))
12 if is_happy_number(num):
13     print(f"{num} is a Happy Number")
14 else:
15     print(f"{num} is not a Happy Number")
```

Enter a number: 23

23 is a Happy Number

## Program 46

Write a Python program to print all happy numbers between 1 and 100.

```
In [4]: 1 def is_happy_number(num):
2         seen = set()
3
4         while num != 1 and num not in seen:
5             seen.add(num)
6             num = sum(int(i) ** 2 for i in str(num))
7
8         return num == 1
9
10 happy_numbers = []
11
12 for num in range(1, 101):
13     if is_happy_number(num):
14         happy_numbers.append(num)
15
16 print("Happy Numbers between 1 and 100:")
17 print(happy_numbers)
```

Happy Numbers between 1 and 100:

[1, 7, 10, 13, 19, 23, 28, 31, 32, 44, 49, 68, 70, 79, 82, 86, 91, 94, 97, 100]

## Program 47

Write a Python program to determine whether the given number is a Harshad Number.

A **Harshad number** (or Niven number) is an integer that is divisible by the sum of its digits. In other words, a number is considered a Harshad number if it can be evenly divided by the sum of its own digits.

For example:

- 18 is a Harshad number because  $1 + 8 = 9$ , and 18 is divisible by 9
- 42 is not a Harshad number because  $4 + 2 = 6$ , and 42 is not divisible by 6.

```
In [5]: 1 def is_harshad_number(num):
2         # Calculate the sum of the digits of the number
3         digit_sum = sum(int(i) for i in str(num))
4
5         # Check if the number is divisible by the sum of its digits
6         return num % digit_sum == 0
7
8     # Input a number
9     num = int(input("Enter a number: "))
10
11    # Check if it's a Harshad Number
12    if is_harshad_number(num):
13        print(f"{num} is a Harshad Number.")
14    else:
15        print(f"{num} is not a Harshad Number.")
```

Enter a number: 18

18 is a Harshad Number.

## Program 48

**Write a Python program to print all pronic numbers between 1 and 100.**

A pronic number, also known as an oblong number or rectangular number, is a type of figurate number that represents a rectangle. It is the product of two consecutive integers,  $n$  and  $(n + 1)$ . Mathematically, a pronic number can be expressed as:

$$P_n = n * (n + 1)$$

For example, the first few pronic numbers are:

- $P_1 = 1 * (1 + 1) = 2$
- $P_2 = 2 * (2 + 1) = 6$
- $P_3 = 3 * (3 + 1) = 12$
- $P_4 = 4 * (4 + 1) = 20$

```
In [6]: 1 def is_pronic_number(num):
2         for n in range(1, int(num**0.5) + 1):
3             if n * (n + 1) == num:
4                 return True
5         return False
6
7     print("Pronic numbers between 1 and 100 are:")
8     for i in range(1, 101):
9         if is_pronic_number(i):
10             print(i, end=" | ")
11
```

Pronic numbers between 1 and 100 are:

2 | 6 | 12 | 20 | 30 | 42 | 56 | 72 | 90 |

## Program 49



Write a Python program to find sum of elements in list

```
In [7]: 1 # Sample List of numbers
2 numbers = [10, 20, 30, 40, 50]
3
4 # Initialize a variable to store the sum
5 sum_of_numbers = 0
6
7 # Iterate through the list and accumulate the sum
8 for i in numbers:
9     sum_of_numbers += i
10
11 # Print the sum
12 print("Sum of elements in the list:", sum_of_numbers)
```

Sum of elements in the list: 150

## Program 50

Write a Python program to Multiply all numbers in the list.

```
In [8]: 1 # Sample List of numbers
2 numbers = [10, 20, 30, 40, 50]
3
4 # Initialize a variable to store the product
5 product_of_numbers = 1
6
7 # Iterate through the list and accumulate the product
8 for i in numbers:
9     product_of_numbers *= i
10
11 # Print the product
12 print("Product of elements in the list:", product_of_numbers)
```

Product of elements in the list: 12000000

## Program 51

Write a Python program to find smallest number in a list.

```
In [9]: 1 # Sample List of numbers
2 numbers = [30, 10, -45, 5, 20]
3
4 # Initialize a variable to store the minimum value, initially set to the first element
5 minimum = numbers[0]
6
7 # Iterate through the list and update the minimum value if a smaller number is found
8 for i in numbers:
9     if i < minimum:
10         minimum = i
11
12 # Print the minimum value
13 print("The smallest number in the list is:", minimum)
```

The smallest number in the list is: -45

## Program 52

Write a Python program to find largest number in a list.

```
In [10]: 1 # Sample List of numbers
2 numbers = [30, 10, -45, 5, 20]
3
4 # Initialize a variable to store the minimum value, initially set to the first element
5 minimum = numbers[0]
6
7 # Iterate through the list and update the minimum value if a smaller number is found
8 for i in numbers:
9     if i > minimum:
10         minimum = i
11
12 # Print the minimum value
13 print("The largest number in the list is:", minimum)
```

The largest number in the list is: 30

## Program 53

Write a Python program to find second largest number in a list.

```
In [11]: 1 # Sample List of numbers
2 numbers = [30, 10, 45, 5, 20]
3
4 # Sort the list in descending order
5 numbers.sort(reverse=True)
6
7 # Check if there are at least two elements in the list
8 if len(numbers) >= 2:
9     second_largest = numbers[1]
10     print("The second largest number in the list is:", second_largest)
11 else:
12     print("The list does not contain a second largest number.")
```

The second largest number in the list is: 30

## Program 54

Write a Python program to find N largest elements from a list.

```
In [12]: 1 def find_n_largest_elements(lst, n):
2         # Sort the list in descending order
3         sorted_lst = sorted(lst, reverse=True)
4
5         # Get the first N elements
6         largest_elements = sorted_lst[:n]
7
8         return largest_elements
9
10 # Sample List of numbers
11 numbers = [30, 10, 45, 5, 20, 50, 15, 3, 345, 54, 67, 87, 98, 100, 34,
12
13 # Number of largest elements to find
14 N = int(input("N = "))
15
16 # Find the N largest elements from the list
17 result = find_n_largest_elements(numbers, N)
18
19 # Print the N largest elements
20 print(f"The {N} largest elements in the list are:", result)
```

N = 3

The 3 largest elements in the list are: [345, 100, 98]

## Program 55

Write a Python program to print even numbers in a list.

```
In [13]: 1 # Sample List of numbers
2 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
3
4 # Using a list comprehension to filter even numbers
5 even_numbers = [num for num in numbers if num % 2 == 0]
6
7 # Print the even numbers
8 print("Even numbers in the list:", even_numbers)
```

Even numbers in the list: [2, 4, 6, 8, 10]

## Program 56

Write a Python program to print odd numbers in a List.

```
In [14]: 1 # Sample List of numbers
2 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
3
4 # Using a list comprehension to filter even numbers
5 even_numbers = [num for num in numbers if num % 2 != 0]
6
7 # Print the even numbers
8 print("Odd numbers in the list:", even_numbers)
```

Odd numbers in the list: [1, 3, 5, 7, 9]

## Program 57

Write a Python program to Remove empty List from List.

```
In [15]: 1 # Sample List containing Lists
2 list_of_lists = [[1, 2, 3], [], [4, 5], [], [6, 7, 8], []]
3
4 # Using a list comprehension to remove empty lists
5 filtered_list = [i for i in list_of_lists if i]
6
7 # Print the filtered List
8 print("List after removing empty lists:", filtered_list)
```

List after removing empty lists: [[1, 2, 3], [4, 5], [6, 7, 8]]

## Program 58

Write a Python program to Cloning or Copying a list.

```
In [16]: 1 # 1. Using Using the Slice Operator
2 original_list = [1, 2, 3, 4, 5]
3 cloned_list = original_list[:]
4 print(cloned_list)
```

[1, 2, 3, 4, 5]

```
In [17]: 1 # 2. Using the list() constructor
2 original_list = [1, 2, 3, 4, 5]
3 cloned_list = list(original_list)
4 print(cloned_list)
```

[1, 2, 3, 4, 5]

```
In [18]: 1 # 3. Using List Comprehension
2 original_list = [1, 2, 3, 4, 5]
3 cloned_list = [item for item in original_list]
4 print(cloned_list)
```

[1, 2, 3, 4, 5]

## Program 59

Write a Python program to Count occurrences of an element in a list.

```
In [19]: 1 def count_occurrences(l, element):
2         count = l.count(element)
3         return count
4
5 # Example usage:
6 my_list = [1, 2, 3, 4, 2, 5, 2, 3, 4, 6, 5]
7 element_to_count = 2
8
9 occurrences = count_occurrences(my_list, element_to_count)
10 print(f"The element {element_to_count} appears {occurrences} times in the list.")
```

The element 2 appears 3 times in the list.

## Program 60

Write a Python program to find words which are greater than given length k.

```
In [20]: 1 def find_words(words, k):
2         # Create an empty list to store words greater than k
3         result = []
4
5         # Iterate through each word in the list
6         for i in words:
7             # Check if the length of the i is greater than k
8             if len(i) > k:
9                 # If yes, append it to the result list
10                result.append(i)
11
12        return result
13
14 # Example usage
15 word_list = ["apple", "banana", "cherry", "date", "elderberry", "dragon fruit"]
16 k = 5
17 long_words = find_words(word_list, k)
18
19 print(f"Words longer than {k} characters: {long_words}")
```

Words longer than 5 characters: ['banana', 'cherry', 'elderberry', 'dragon fruit']

## Program 61

Write a Python program for removing  $i^{th}$  character from a string.

```
In [21]: 1 def remove_char(input_str, i):
2         # Check if i is a valid index
3         if i < 0 or i >= len(input_str):
4             print(f"Invalid index {i}. The string remains unchanged.")
5             return input_str
6
7         # Remove the i-th character using slicing
8         result_str = input_str[:i] + input_str[i + 1:]
9
10        return result_str
11
12        # Input string
13        input_str = "Hello, wWorld!"
14        i = 7 # Index of the character to remove
15
16        # Remove the i-th character
17        new_str = remove_char(input_str, i)
18
19        print(f"Original String: {input_str}")
20        print(f"String after removing {i}th character : {new_str}")
```

Original String: Hello, wWorld!

String after removing 7th character : Hello, World!

## Program 62

Write a Python program to split and join a string.

```
In [22]: 1 # Split a string into a list of words
2         input_str = "Python program to split and join a string"
3         word_list = input_str.split() # By default, split on whitespace
4
5         # Join the list of words into a string
6         separator = " " # specify the separator between words
7         output_str = separator.join(word_list)
8
9         # Print the results
10        print("Original String:", input_str)
11        print("List of split Words:", word_list)
12        print("Joined String:", output_str)
```

Original String: Python program to split and join a string

List of split Words: ['Python', 'program', 'to', 'split', 'and', 'join', 'a', 'string']

Joined String: Python program to split and join a string

## Program 63

Write a Python program to check if a given string is binary string or not.

```
In [23]: 1 def is_binary_str(input_str):
2         # Iterate through each character in the input string
3         for i in input_str:
4             # Check if the i is not '0' or '1'
5             if i not in '01':
6                 return False # If any character is not '0' or '1', it's not a binary string
7         return True # If all characters are '0' or '1', it's a binary string
8
9         # Input string to check
10        input_str = "1001110"
11
12        # Check if the input string is a binary string
13        if is_binary_str(input_str):
14            print(f"'{input_str}' is a binary string.")
15        else:
16            print(f"'{input_str}' is not a binary string.")
```

'1001110' is a binary string.

## Program 64

Write a Python program to find uncommon words from two Strings.

```
In [24]: 1 def uncommon_words(str1, str2):
2         # Split the strings into words and convert them to sets
3         words1 = set(str1.split())
4         words2 = set(str2.split())
5
6         # Find uncommon words by taking the set difference
7         uncommon_words_set = words1.symmetric_difference(words2)
8
9         # Convert the set of uncommon words back to a list
10        uncommon_words_list = list(uncommon_words_set)
11
12        return uncommon_words_list
13
14        # Input two strings
15        string1 = "This is the first string"
16        string2 = "This is the second string"
17
18        # Find uncommon words between the two strings
19        uncommon = uncommon_words(string1, string2)
20
21        # Print the uncommon words
22        print("Uncommon words:", uncommon)
```

Uncommon words: ['second', 'first']

## Program 65

Write a Python program to find all duplicate characters in string.

```
In [25]: 1 def find_duplicates(input_str):
2         # Create an empty dictionary to store character counts
3         char_count = {}
4
5         # Initialize a list to store duplicate characters
6         duplicates = []
7
8         # Iterate through each character in the input string
9         for i in input_str:
10            # If the character is already in the dictionary, increment its
11            if i in char_count:
12                char_count[i] += 1
13            else:
14                char_count[i] = 1
15
16            # Iterate through the dictionary and add characters with count > 1
17            for i, count in char_count.items():
18                if count > 1:
19                    duplicates.append(i)
20
21            return duplicates
22
23        # Input a string
24        input_string = "piyush sharma"
25
26        # Find duplicate characters in the string
27        duplicate_chars = find_duplicates(input_string)
28
29        # Print the list of duplicate characters
30        print("Duplicate characters:", duplicate_chars)
```

Duplicate characters: ['s', 'h', 'a']

## Program 66

Write a Python Program to check if a string contains any special character.



```
In [26]: 1 import re
2
3 def check_special_char(in_str):
4     # Define a regular expression pattern to match special characters
5     pattern = r'[!@#$%^&*()_+{\} \[\]:;<>,.?~\\\/\'\"-=]'
6
7     # Use re.search to find a match in the input string
8     if re.search(pattern, in_str):
9         return True
10    else:
11        return False
12
13    # Input a string
14    input_string = str(input("Enter a string: "))
15
16    # Check if the string contains any special characters
17    contains_special = check_special_char(input_string)
18
19    # Print the result
20    if contains_special:
21        print("The string contains special characters.")
22    else:
23        print("The string does not contain special characters.")
```

Enter a string: "Hello, World!"

The string contains special characters.

## Program 67

Write a Python program to Extract Unique dictionary values.

```
In [27]: 1 # Sample dictionary
2 my_dict = {
3     'a': 10,
4     'b': 20,
5     'c': 10,
6     'd': 30,
7     'e': 20
8 }
9
10 # Initialize an empty set to store unique values
11 uni_val = set()
12
13 # Iterate through the values of the dictionary
14 for i in my_dict.values():
15     # Add each value to the set
16     uni_val.add(i)
17
18 # Convert the set of unique values back to a list (if needed)
19 unique_values_list = list(uni_val)
20
21 # Print the unique values
22 print("Unique values in the dictionary:", unique_values_list)
```

Unique values in the dictionary: [10, 20, 30]

## Program 68

Write a Python program to find the sum of all items in a dictionary.

```
In [28]: 1 # Sample dictionary
2 my_dict = {
3     'a': 10,
4     'b': 20,
5     'c': 30,
6     'd': 40,
7     'e': 50
8 }
9
10 # Initialize a variable to store the sum
11 total_sum = 0
12
13 # Iterate through the values of the dictionary and add them to the total
14 for i in my_dict.values():
15     total_sum += i
16
17 # Print the sum of all items in the dictionary
18 print("Sum of all items in the dictionary:", total_sum)
```

Sum of all items in the dictionary: 150

## Program 69

Write a Python program to Merging two Dictionaries.

```
In [29]: 1 # 1. Using the update() method:
2
3 dict1 = {'a': 1, 'b': 2}
4 dict2 = {'c': 3, 'd': 4}
5
6 dict1.update(dict2)
7
8 # The merged dictionary is now in dict1
9 print("Merged Dictionary (using update()):", dict1)
```

Merged Dictionary (using update()): {'a': 1, 'b': 2, 'c': 3, 'd': 4}

```
In [30]: 1 # 2. Using dictionary unpacking
2
3 dict1 = {'a': 1, 'b': 2}
4 dict2 = {'c': 3, 'd': 4}
5
6 # Merge dict2 into dict1 using dictionary unpacking
7 merged_dict = {**dict1, **dict2}
8
9 # The merged dictionary is now in merged_dict
10 print("Merged Dictionary (using dictionary unpacking):", merged_dict)
```

Merged Dictionary (using dictionary unpacking): {'a': 1, 'b': 2, 'c': 3, 'd': 4}

## Program 70

Write a Python program to convert key-values list to flat dictionary.

```
In [31]: 1 key_values_list = [('a', 1), ('b', 2), ('c', 3), ('d', 4)]
2
3 # Initialize an empty dictionary
4 flat_dict = {}
5
6 # Iterate through the list and add key-value pairs to the dictionary
7 for key, value in key_values_list:
8     flat_dict[key] = value
9
10 # Print the resulting flat dictionary
11 print("Flat Dictionary:", flat_dict)
```

Flat Dictionary: {'a': 1, 'b': 2, 'c': 3, 'd': 4}

## Program 71

Write a Python program to insertion at the beginning in OrderedDict.

```
In [32]: 1 from collections import OrderedDict
2
3 # Create an OrderedDict
4 ordered_dict = OrderedDict([('b', 2), ('c', 3), ('d', 4)])
5
6 # Item to insert at the beginning
7 new_item = ('a', 1)
8
9 # Create a new OrderedDict with the new item as the first element
10 new_ordered_dict = OrderedDict([new_item])
11
12 # Merge the new OrderedDict with the original OrderedDict
13 new_ordered_dict.update(ordered_dict)
14
15 # Print the updated OrderedDict
16 print("Updated OrderedDict:", new_ordered_dict)
```

Updated OrderedDict: OrderedDict([('a', 1), ('b', 2), ('c', 3), ('d', 4)])

## Program 72

Write a Python program to check order of character in string using OrderedDict().



```
In [33]: 1 from collections import OrderedDict
2
3 def check_order(string, reference):
4     # Create OrderedDicts for both strings
5     string_dict = OrderedDict.fromkeys(string)
6     reference_dict = OrderedDict.fromkeys(reference)
7
8     # Check if the OrderedDict for the string matches the OrderedDict for the reference
9     return string_dict == reference_dict
10
11 # Input strings
12 input_string = "hello world"
13 reference_string = "helo wrd"
14
15 # Check if the order of characters in input_string matches reference_string
16 if check_order(input_string, reference_string):
17     print("The order of characters in the input string matches the reference string.")
18 else:
19     print("The order of characters in the input string does not match the reference string.")
20
```

The order of characters in the input string matches the reference string.

## Program 73

Write a Python program to sort Python Dictionaries by Key or Value.

```
In [34]: 1 # Sort by Keys:
2
3 sample_dict = {'apple': 3, 'banana': 1, 'cherry': 2, 'date': 4}
4
5 sorted_dict_by_keys = dict(sorted(sample_dict.items()))
6
7 print("Sorted by keys:")
8 for key, value in sorted_dict_by_keys.items():
9     print(f"{key}: {value}")
```

Sorted by keys:  
apple: 3  
banana: 1  
cherry: 2  
date: 4

```
In [35]: 1 # Sort by values
2
3 sample_dict = {'apple': 3, 'banana': 1, 'cherry': 2, 'date': 4}
4
5 sorted_dict_by_values = dict(sorted(sample_dict.items(), key=lambda item: item[1]))
6
7 print("Sorted by values:")
8 for key, value in sorted_dict_by_values.items():
9     print(f"{key}: {value}")
```

Sorted by values:  
banana: 1  
cherry: 2  
apple: 3  
date: 4

## Program 74

Write a program that calculates and prints the value according to the given formula:

$$Q = \text{Square root of } \frac{2CD}{H}$$

Following are the fixed values of C and H:

C is 50. H is 30.

D is the variable whose values should be input to your program in a comma-separated sequence.

Example

Let us assume the following comma separated input sequence is given to the program:

100,150,180

The output of the program should be:

18,22,24

```
In [36]: 1 import math
2
3 # Fixed values
4 C = 50
5 H = 30
6
7 # Function to calculate Q
8 def calculate_Q(D):
9     return int(math.sqrt((2 * C * D) / H))
10
11 # Input comma-separated sequence of D values
12 input_sequence = input("Enter comma-separated values of D: ")
13 D_values = input_sequence.split(',')
14
15 # Calculate and print Q for each D value
16 result = [calculate_Q(int(D)) for D in D_values]
17 print(', '.join(map(str, result)))
```

Enter comma-separated values of D: 100,150,180  
18,22,24

## Program 75

Write a program which takes 2 digits, X,Y as input and generates a 2-dimensional array. The element value in the i-th row and j-th column of the array should be  $i*j$ .

Note:  $i=0,1,.., X-1$ ;  $j=0,1,..,Y-1$ .

Example

Suppose the following inputs are given to the program:

3,5

Then, the output of the program should be:

`[[0, 0, 0, 0, 0], [0, 1, 2, 3, 4], [0, 2, 4, 6, 8]]`

```
In [37]: 1 # Input two digits, X and Y
2 X, Y = map(int, input("Enter two digits (X, Y): ").split(','))
3
4 # Initialize a 2D array filled with zeros
5 array = [[0 for j in range(Y)] for i in range(X)]
6
7 # Fill the array with values i * j
8 for i in range(X):
9     for j in range(Y):
10         array[i][j] = i * j
11
12 # Print the 2D array
13 for row in array:
14     print(row)
```

Enter two digits (X, Y): 3,5  
[0, 0, 0, 0, 0]  
[0, 1, 2, 3, 4]  
[0, 2, 4, 6, 8]

## Program 76

Write a program that accepts a comma separated sequence of words as input and prints the words in a comma-separated sequence after sorting them alphabetically.

Suppose the following input is supplied to the program:

without,hello,bag,world

Then, the output should be:

bag,hello,without,world

```
In [38]: 1 # Accept input from the user
2 input_sequence = input("Enter a comma-separated sequence of words: ")
3
4 # Split the input into a list of words
5 words = input_sequence.split(',')
6
7 # Sort the words alphabetically
8 sorted_words = sorted(words)
9
10 # Join the sorted words into a comma-separated sequence
11 sorted_sequence = ','.join(sorted_words)
12
13 # Print the sorted sequence
14 print("Sorted words:", sorted_sequence)
```

Enter a comma-separated sequence of words: without, hello, bag, world  
Sorted words: bag, hello, world,without

## Program 77

Write a program that accepts a sequence of whitespace separated words as input and prints the words after removing all duplicate words and sorting them alphanumerically.

Suppose the following input is supplied to the program:

hello world and practice makes perfect and hello world again

Then, the output should be:

again and hello makes perfect practice world

```
In [39]: 1 # Accept input from the user
2 input_sequence = input("Enter a sequence of whitespace-separated words: ")
3
4 # Split the input into words and convert it into a set to remove duplicates
5 words = set(input_sequence.split())
6
7 # Sort the unique words alphanumerically
8 sorted_words = sorted(words)
9
10 # Join the sorted words into a string with whitespace separation
11 result = ' '.join(sorted_words)
12
13 # Print the result
14 print("Result:", result)
```

Enter a sequence of whitespace-separated words: hello world and practice makes perfect and hello world again  
Result: again and hello makes perfect practice world

## Program 79

Write a program that accepts a sentence and calculate the number of letters and digits. Suppose the following input is supplied to the program:

hello world! 123

Then, the output should be:

LETTERS 10

DIGITS 3

```
In [40]: 1 # Accept input from the user
2 sentence = input("Enter a sentence: ")
3
4 # Initialize counters for letters and digits
5 letter_count = 0
6 digit_count = 0
7
8 # Iterate through each character in the sentence
9 for char in sentence:
10     if char.isalpha():
11         letter_count += 1
12     elif char.isdigit():
13         digit_count += 1
14
15 # Print the results
16 print("LETTERS", letter_count)
17 print("DIGITS", digit_count)
```

Enter a sentence: hello world! 123  
LETTERS 10  
DIGITS 3



## Program 80

A website requires the users to input username and password to register. Write a program to check the validity of password input by users. Following are the criteria for checking the password:

1. At least 1 letter between [a-z]
2. At least 1 number between [0-9]
1. At least 1 letter between [A-Z]
3. At least 1 character from [\$#@]
4. Minimum length of transaction password: 6
5. Maximum length of transaction password: 12

Your program should accept a sequence of comma separated passwords and will check them according to the above criteria. Passwords that match the criteria are to be printed, each separated by a comma.

### Example

If the following passwords are given as input to the program:

ABd1234@1,a F1#,2w3E\*,2We3345

Then, the output of the program should be:

ABd1234@1

```
In [41]: 1 import re
2
3 # Function to check if a password is valid
4 def is_valid_password(password):
5     # Check the length of the password
6     if 6 <= len(password) <= 12:
7         # Check if the password matches all the criteria using regular
8         if re.match(r"^(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9])(?=.*[$#@])", password):
9             return True
10        return False
11
12 # Accept input from the user as comma-separated passwords
13 passwords = input("Enter passwords separated by commas: ").split(',')
14
15 # Initialize a list to store valid passwords
16 valid_passwords = []
17
18 # Iterate through the passwords and check their validity
19 for psw in passwords:
20     if is_valid_password(psw):
21         valid_passwords.append(psw)
22
23 # Print the valid passwords separated by commas
24 print(','.join(valid_passwords))
```

Enter passwords separated by commas: ABd1234@1,a F1#,2w3E\*,2We3345  
ABd1234@1

## Program 81

Define a class with a generator which can iterate the numbers, which are divisible by 7, between a given range 0 and n.

```
In [42]: 1 class DivisibleBySeven:
2         def __init__(self,n):
3             self.n = n
4
5         def generate_divisible_by_seven(self):
6             for num in range(self.n + 1):
7                 if num%7 == 0:
8                     yield num
```

```
In [43]: 1 n = int(input("Enter your desired range: "))
2
3 divisible_by_seven_generator = DivisibleBySeven(n).generate_divisible_b
4
5 for num in divisible_by_seven_generator:
6     print(num)
```

Enter your desired range: 50

0

7

14

21

28

35

42

49

## Program 82

Write a program to compute the frequency of the words from the input. The output should output after sorting the key alphanumerically. Suppose the following input is supplied to the program:

New to Python or choosing between Python 2 and Python 3? Read Python 2 or Python 3.

Then, the output should be:

2:2

3.:1

3?:1

New:1

Python:5

Read:1

and:1

between:1

choosing:1

or:2

to:1

```
In [44]: 1 input_sentence = input("Enter a sentence: ")
2
3 # Split the sentence into words
4 words = input_sentence.split()
5
6 # Create a dictionary to store word frequencies
7 word_freq = {}
8
9 # Count word frequencies
10 for word in words:
11     # Remove punctuation (., ?) from the word
12     word = word.strip('.,?')
13     # Convert the word to lowercase to ensure case-insensitive counting
14     word = word.lower()
15     # Update the word frequency in the dictionary
16     if word in word_freq:
17         word_freq[word] += 1
18     else:
19         word_freq[word] = 1
20
21 # Sort the words alphanumerically
22 sorted_words = sorted(word_freq.items())
23
24 # Print the word frequencies
25 for word, frequency in sorted_words:
26     print(f"{word}:{frequency}")
```

Enter a sentence: New to Python or choosing between Python 2 and Python 3?  
Read Python 2 or Python 3.

2:2  
3:2  
and:1  
between:1  
choosing:1  
new:1  
or:2  
python:5  
read:1  
to:1

## Program 83

**Define a class Person and its two child classes: Male and Female. All classes have a method "getGender" which can print "Male" for Male class and "Female" for Female class.**

```
In [45]: 1 class Person:
2         def getGender(self):
3             return "Unknown"
4
5         class Male(Person):
6             def getGender(self):
7                 return "Male"
8
9         class Female(Person):
10            def getGender(self):
11                return "Female"
```

```
In [46]: 1 person = Person()
2         male = Male()
3         female = Female()
4
5         print(person.getGender())
6         print(male.getGender())
7         print(female.getGender())
```

Unknown  
Male  
Female

## Program 84

Please write a program to generate all sentences where subject is in ["I", "You"] and verb is in ["Play", "Love"] and the object is in ["Hockey", "Football"].

```
In [47]: 1 subjects = ["I", "You"]
2         verbs = ["Play", "Love"]
3         objects = ["Hockey", "Football"]
4
5         sentences = []
6
7         for sub in subjects:
8             for vrb in verbs:
9                 for obj in objects:
10                    sentence = f"{sub} {vrb} {obj}."
11                    sentences.append(sentence)
12
13         for sentence in sentences:
14             print(sentence)
```

I Play Hockey.  
I Play Football.  
I Love Hockey.  
I Love Football.  
You Play Hockey.  
You Play Football.  
You Love Hockey.  
You Love Football.

## Program 85

Please write a program to compress and decompress the string "hello world!hello world!hello world!hello world!".

In [48]:

```
1 import zlib
2
3 string = "hello world!hello world!hello world!hello world!"
4
5 # Compress the string
6 compressed_string = zlib.compress(string.encode())
7
8 # Decompress the string
9 decompressed_string = zlib.decompress(compressed_string).decode()
10
11 print("Original String:", string)
12 print("Compressed String:", compressed_string)
13 print("Decompressed String:", decompressed_string)
```

Original String: hello world!hello world!hello world!hello world!

Compressed String: b'x\x9c\xcbH\xcd\x09\x09W(\xcf/\xcaIQ\xcc \x82\r\x00\xbd[\x11\xf5'

Decompressed String: hello world!hello world!hello world!hello world!

## Program 86

Please write a binary search function which searches an item in a sorted list. The function should return the index of element to be searched in the list.

```
In [49]: 1 def binary_search(arr, target):
2         left, right = 0, len(arr) - 1
3
4         while left <= right:
5             mid = (left + right) // 2
6
7             if arr[mid] == target:
8                 return mid # Element found, return its index
9             elif arr[mid] < target:
10                left = mid + 1 # Target is in the right half
11            else:
12                right = mid - 1 # Target is in the left half
13
14        return -1 # Element not found in the list
15
16 # Example usage:
17 sorted_list = [1, 2, 3, 4, 5, 6, 7, 8, 9]
18 target_element = 4
19
20 result = binary_search(sorted_list, target_element)
21
22 if result != -1:
23     print(f"Element {target_element} found at index {result}")
24 else:
25     print(f"Element {target_element} not found in the list")
```

Element 4 found at index 3

## Program 87

Please write a program using generator to print the numbers which can be divisible by 5 and 7 between 0 and n in comma separated form while n is input by console.

Example:

If the following n is given as input to the program:

100

Then, the output of the program should be:

0,35,70

```
In [50]: 1 def divisible_by_5_and_7(n):
2         for num in range(n + 1):
3             if num % 5 == 0 and num % 7 == 0:
4                 yield num
```

```
In [51]: 1 try:
2         n = int(input("Enter a value for n: "))
3         result = divisible_by_5_and_7(n)
4         print(','.join(map(str, result)))
5     except ValueError:
6         print("Invalid input. Please enter a valid integer for n.")
```

Enter a value for n: 100

0,35,70

## Program 88

Please write a program using generator to print the even numbers between 0 and n in comma separated form while n is input by console.

Example:

If the following n is given as input to the program:

10

Then, the output of the program should be:

0,2,4,6,8,10

In [52]:

```
1 def even_numbers(n):
2     for num in range(n + 1):
3         if num % 2 == 0:
4             yield num
```

In [53]:

```
1 try:
2     n = int(input("Enter a value for n: "))
3     result = even_numbers(n)
4     print(','.join(map(str, result)))
5 except ValueError:
6     print("Invalid input. Please enter a valid integer for n.")
```

Enter a value for n: 10

0,2,4,6,8,10

## Program 89

The Fibonacci Sequence is computed based on the following formula:

$f(n)=0$  if  $n=0$

$f(n)=1$  if  $n=1$

$f(n)=f(n-1)+f(n-2)$  if  $n>1$

Please write a program using list comprehension to print the Fibonacci Sequence in comma separated form with a given n input by console.

Example:

If the following n is given as input to the program:

8

Then, the output of the program should be:

0,1,1,2,3,5,8,13



```
In [55]: 1 def fibonacci(n):
2         sequence = [0, 1] # Initializing the sequence with the first two F
3         [sequence.append(sequence[-1] + sequence[-2]) for _ in range(2, n)]
4         return sequence
```

```
In [56]: 1 try:
2         n = int(input("Enter a value for n: "))
3         result = fibonacci(n)
4         print(','.join(map(str, result)))
5     except ValueError:
6         print("Invalid input. Please enter a valid integer for n.")
```

Enter a value for n: 8  
0,1,1,2,3,5,8,13

## Program 90

Assuming that we have some email addresses in the `"username@companyname.com (mailto:username@companyname.com)"` format, please write program to print the user name of a given email address. Both user names and company names are composed of letters only.

Example:

If the following email address is given as input to the program:

[john@google.com \(mailto:john@google.com\)](mailto:john@google.com)

Then, the output of the program should be:

john

```
In [57]: 1 def extract_username(email):
2         # Split the email address at '@' to separate the username and domain
3         parts = email.split('@')
4
5         # Check if the email address has the expected format
6         if len(parts) == 2:
7             return parts[0] # The username is the first part
8         else:
9             return "Invalid email format"
```

```
In [58]: 1 try:
2         email = input("Enter an email address: ")
3         username = extract_username(email)
4         print(username)
5     except ValueError:
6         print("Invalid input. Please enter a valid email address.")
```

Enter an email address: john@google.com  
john

## Program 91

Define a class named Shape and its subclass Square. The Square class has an init function which takes a length as argument. Both classes have an area function which can print the area of the shape where Shape's area is 0 by default.

In [59]:

```

1  class Shape:
2      def __init__(self):
3          pass # Default constructor, no need to initialize anything
4
5      def area(self):
6          return 0 # Shape's area is 0 by default
7
8
9  class Square(Shape):
10     def __init__(self, length):
11         super().__init__() # Call the constructor of the parent class
12         self.length = length
13
14     def area(self):
15         return self.length ** 2 # Calculate the area of the square

```

In [60]:

```

1  # Create instances of the classes
2  shape = Shape()
3  square = Square(float(input("Enter the shape of the square: ")))
4
5  # Calculate and print the areas
6  print("Shape's area by default:", shape.area())
7  print("Area of the square:", square.area())

```

Enter the shape of the square: 5  
 Shape's area by default: 0  
 Area of the square: 25.0

## Program 92

Write a function that stutters a word as if someone is struggling to read it. The first two letters are repeated twice with an ellipsis ... and space after each, and then the word is pronounced with a question mark ?.

Examples

stutter("incredible") → "in... in... incredible?"

stutter("enthusiastic") → "en... en... enthusiastic?"

stutter("outstanding") → "ou... ou... outstanding?"

Hint :- Assume all input is in lower case and at least two characters long.

```
In [61]: 1 def stutter(word):
2         if len(word) < 2:
3             return "Word must be at least two characters long."
4
5         stuttered_word = f"{word[:2]}... {word[:2]}... {word}?"
6         return stuttered_word
```

```
In [62]: 1 # Test cases
2 print(stutter("incredible"))
3 print(stutter("enthusiastic"))
4 print(stutter("outstanding"))
```

```
in... in... incredible?
en... en... enthusiastic?
ou... ou... outstanding?
```

## Program 93

Create a function that takes an angle in radians and returns the corresponding angle in degrees rounded to one decimal place.

Examples

radians\_to\_degrees(1) → 57.3

radians\_to\_degrees(20) → 1145.9

radians\_to\_degrees(50) → 2864.8

```
In [63]: 1 import math
2
3 def radians_to_degrees(radians):
4     degrees = radians * (180 / math.pi)
5     return round(degrees, 1)
```

```
In [64]: 1 # Test cases
2 print(radians_to_degrees(1))
3 print(radians_to_degrees(20))
4 print(radians_to_degrees(50))
```

```
57.3
1145.9
2864.8
```

## Program 94

In this challenge, establish if a given integer num is a Curzon number. If 1 plus 2 elevated to num is exactly divisible by 1 plus 2 multiplied by num, then num is a Curzon number.

Given a non-negative integer num, implement a function that returns True if num is a Curzon number, or False otherwise.

Examples

**is\_curzon(5) → True**

#  $2^{**} 5 + 1 = 33$

#  $2 * 5 + 1 = 11$

# 33 is a multiple of 11

**is\_curzon(10) → False**

#  $2^{**} 10 + 1 = 1025$

#  $2 * 10 + 1 = 21$

# 1025 is not a multiple of 21

**is\_curzon(14) → True**

#  $2^{**} 14 + 1 = 16385$

#  $2 * 14 + 1 = 29$

# 16385 is a multiple of 29

### Curzon Number:

It is defined based on a specific mathematical relationship involving powers of 2. An integer 'n' is considered a Curzon number if it satisfies the following condition:

*If  $(2^n + 1)$  is divisible by  $(2n + 1)$ , then 'n' is a Curzon number.*

For example:

- If  $n = 5$ :  $2^5 + 1 = 33$ , and  $2 * 5 + 1 = 11$ . Since 33 is divisible by 11 ( $33 \% 11 = 0$ ), 5 is a Curzon number.
- If  $n = 10$ :  $2^{10} + 1 = 1025$ , and  $2 * 10 + 1 = 21$ . 1025 is not divisible by 21, so 10 is not a Curzon number.

Curzon numbers are a specific **subset of integers** with this unique mathematical property.

In [65]:

```
1 def is_curzon(num):
2     numerator = 2 ** num + 1
3     denominator = 2 * num + 1
4     return numerator % denominator == 0
```

In [66]:

```
1 # Test cases
2 print(is_curzon(5))
3 print(is_curzon(10))
4 print(is_curzon(14))
```

True  
False  
True

## Program 95

Given the side length x find the area of a hexagon.

Examples

area\_of\_hexagon(1) → 2.6

area\_of\_hexagon(2) → 10.4

area\_of\_hexagon(3) → 23.4

```
In [67]: 1 import math
          2
          3 def area_of_hexagon(x):
          4     area = (3 * math.sqrt(3) * x**2) / 2
          5     return round(area, 1)
```

```
In [68]: 1 # Example usage:
          2 print(area_of_hexagon(1))
          3 print(area_of_hexagon(2))
          4 print(area_of_hexagon(3))
```

2.6

10.4

23.4

## Program 96

Create a function that returns a base-2 (binary) representation of a base-10 (decimal) string number. To convert is simple: ((2) means base-2 and (10) means base-10)

010101001(2) = 1 + 8 + 32 + 128.

Going from right to left, the value of the most right bit is 1, now from that every bit to the left will be x2 the value, value of an 8 bit binary numbers are (256, 128, 64, 32, 16, 8, 4, 2, 1).

Examples

binary(1) → "1"

# 1\*1 = 1

binary(5) → "101"

# 1\*1 + 1\*4 = 5

binary(10) → 1010

# 1\*2 + 1\*8 = 10

```
In [69]: 1 def binary(decimal):
2         binary_str = ""
3         while decimal > 0:
4             remainder = decimal % 2
5             binary_str = str(remainder) + binary_str
6             decimal = decimal // 2
7         return binary_str if binary_str else "0"
```

```
In [70]: 1 print(binary(1))
2         print(binary(5))
3         print(binary(10))
```

```
1
101
1010
```

## Program 97

Create a function that takes three arguments a, b, c and returns the sum of the numbers that are evenly divided by c from the range a, b inclusive.

### Examples

`evenly_divisible(1, 10, 20) → 0`

# No number between 1 and 10 can be evenly divided by 20.

`evenly_divisible(1, 10, 2) → 30`

# 2 + 4 + 6 + 8 + 10 = 30

`evenly_divisible(1, 10, 3) → 18`

# 3 + 6 + 9 = 18

```
In [71]: 1 def evenly_divisible(a, b, c):
2         total = 0
3         for num in range(a, b + 1):
4             if num % c == 0:
5                 total += num
6         return total
```

```
In [72]: 1 print(evenly_divisible(1, 10, 20))
2         print(evenly_divisible(1, 10, 2))
3         print(evenly_divisible(1, 10, 3))
```

```
0
30
18
```

## Program 98

Create a function that returns True if a given inequality expression is correct and False otherwise.

## Examples

**correct\_signs("3 < 7 < 11") → True**

**correct\_signs("13 > 44 > 33 < 1") → False**

**correct\_signs("1 < 2 < 6 < 9 > 3") → True**

```
In [74]: 1 def correct_signs(expression):
        2     try:
        3         return eval(expression)
        4     except:
        5         return False
```

```
In [75]: 1 print(correct_signs("3 < 7 < 11"))
        2 print(correct_signs("13 > 44 > 33 < 1"))
        3 print(correct_signs("1 < 2 < 6 < 9 > 3"))
```

True  
False  
True

## Program 99

Create a function that replaces all the vowels in a string with a specified character.

### Examples

**replace\_vowels("the aardvark", "#") → "th# ##rdv#rk"**

**replace\_vowels("minnie mouse", "?") → "m?nn?? m??s?"**

**replace\_vowels("shakespeare", "\*") → "shkspr\*"**

```
In [76]: 1 def replace_vowels(string, char):
        2     vowels = "AEIOUaeiou" # List of vowels to be replaced
        3     for vowel in vowels:
        4         string = string.replace(vowel, char) # Replace each vowel with
        5     return string
```

```
In [77]: 1 print(replace_vowels("the aardvark", "#"))
        2 print(replace_vowels("minnie mouse", "?"))
        3 print(replace_vowels("shakespeare", "*"))
```

th# ##rdv#rk  
m?nn?? m??s?  
sh\*k\*sp\*\*r\*

## Program 100

Write a function that calculates the factorial of a number recursively.

### Examples

**factorial(5) → 120**

**factorial(3) → 6**

**factorial(1) → 1**

**factorial(0) → 1**

```
In [78]: 1 def factorial(n):  
2         if n == 0:  
3             return 1 # Base case: factorial of 0 is 1  
4         else:  
5             return n * factorial(n - 1) # Recursive case: n! = n * (n-1)!
```

```
In [79]: 1 print(factorial(5))  
2 print(factorial(3))  
3 print(factorial(1))  
4 print(factorial(0))
```

120

6

1

1

## Program 101

Hamming distance is the number of characters that differ between two strings.

To illustrate:

String1: "abcbba"

String2: "abcbda"

Hamming Distance: 1 - "b" vs. "d" is the only difference.

Create a function that computes the hamming distance between two strings.

Examples

hamming\_distance("abcde", "bcdef") → 5

hamming\_distance("abcde", "abcde") → 0

hamming\_distance("strong", "strung") → 1



```
In [80]: 1 def hamming_distance(str1, str2):
2         # Check if the strings have the same length
3         if len(str1) != len(str2):
4             raise ValueError("Input strings must have the same length")
5
6         # Initialize a counter to keep track of differences
7         distance = 0
8
9         # Iterate through the characters of both strings
10        for i in range(len(str1)):
11            if str1[i] != str2[i]:
12                distance += 1 # Increment the counter for differences
13
14        return distance
```

```
In [81]: 1 print(hamming_distance("abcde", "bcdef"))
2         print(hamming_distance("abcde", "abcde"))
3         print(hamming_distance("strong", "strung"))

5
0
1
```

## Program 102

Create a function that takes a list of non-negative integers and strings and return a new list without the strings.

### Examples

`filter_list([1, 2, "a", "b"]) → [1, 2]`

`filter_list([1, "a", "b", 0, 15]) → [1, 0, 15]`

`filter_list([1, 2, "aasf", "1", "123", 123]) → [1, 2, 123]`

```
In [82]: 1 def filter_list(lst):
2         # Initialize an empty list to store non-string elements
3         result = []
4
5         # Iterate through the elements in the input list
6         for element in lst:
7             # Check if the element is a non-negative integer (not a string)
8             if isinstance(element, int) and element >= 0:
9                 result.append(element)
10
11        return result
```

```
In [83]: 1 filter_list([1, 2, "a", "b"])
```

Out[83]: [1, 2]

```
In [84]: 1 filter_list([1, "a", "b", 0, 15])
```

Out[84]: [1, 0, 15]

```
In [85]: 1 filter_list([1, 2, "aasf", "1", "123", 123])
```

```
Out[85]: [1, 2, 123]
```

## Program 103

The "Reverser" takes a string as input and returns that string in reverse order, with the opposite case.

Examples

`reverse("Hello World")` → "DLROw OLLEh"

`reverse("ReVeRsE")` → "eSrEvEr"

`reverse("Radar")` → "RADAr"

```
In [86]: 1 def reverse(input_str):  
2         # Reverse the string and swap the case of characters  
3         reversed_str = input_str[::-1].swapcase()  
4  
5         return reversed_str
```

```
In [87]: 1 reverse("Hello World")
```

```
Out[87]: 'DLROw OLLEh'
```

```
In [88]: 1 reverse("ReVeRsE")
```

```
Out[88]: 'eSrEvEr'
```

```
In [89]: 1 reverse("Radar")
```

```
Out[89]: 'RADAr'
```

## Program 104

You can assign variables from lists like this:

```
lst = [1, 2, 3, 4, 5, 6]
```

```
first = lst[0]
```

```
middle = lst[1:-1]
```

```
last = lst[-1]
```

```
print(first) → outputs 1
```

```
print(middle) → outputs [2, 3, 4, 5]
```

```
print(last) → outputs 6
```

With Python 3, you can assign variables from lists in a much more succinct way. Create variables first, middle and last from the given list using destructuring assignment (check the Resources tab for some examples), where:

first → 1

middle → [2, 3, 4, 5]

last → 6

Your task is to unpack the list `wriateyourcodehere` into three variables, being

```
In [90]: 1 wriateyourcodehere = [1, 2, 3, 4, 5, 6]
        2
        3 # Unpack the list into variables
        4 first, *middle, last = wriateyourcodehere
```

```
In [91]: 1 first
```

```
Out[91]: 1
```

```
In [92]: 1 middle
```

```
Out[92]: [2, 3, 4, 5]
```

```
In [93]: 1 last
```

```
Out[93]: 6
```

## Program 105

Write a function that calculates the factorial of a number recursively.

Examples

factorial(5) → 120

factorial(3) → 6

factorial(1) → 1

factorial(0) → 1

```
In [94]: 1 def factorial(n):
        2     if n == 0:
        3         return 1 # Base case: factorial of 0 is 1
        4     else:
        5         return n * factorial(n - 1) # Recursive case: n! = n * (n-1)!
```

```
In [95]: 1 factorial(5)
```

```
Out[95]: 120
```

```
In [96]: 1 factorial(3)
```

```
Out[96]: 6
```

```
In [97]: 1 factorial(1)
```

```
Out[97]: 1
```

```
In [98]: 1 factorial(0)
```

```
Out[98]: 1
```

## Program 106

Write a function that moves all elements of one type to the end of the list.

Examples

`move_to_end([1, 3, 2, 4, 4, 1], 1) → [3, 2, 4, 4, 1, 1]`

Move all the 1s to the end of the array.

`move_to_end([7, 8, 9, 1, 2, 3, 4], 9) → [7, 8, 1, 2, 3, 4, 9]`

`move_to_end(["a", "a", "a", "b"], "a") → ["b", "a", "a", "a"]`

```
In [99]: 1 def move_to_end(lst, element):
2         # Initialize a count for the specified element
3         count = lst.count(element)
4
5         # Remove all occurrences of the element from the list
6         lst = [x for x in lst if x != element]
7
8         # Append the element to the end of the list count times
9         lst.extend([element] * count)
10
11        return lst
```

```
In [100]: 1 move_to_end([1, 3, 2, 4, 4, 1], 1)
```

```
Out[100]: [3, 2, 4, 4, 1, 1]
```

```
In [101]: 1 move_to_end([7, 8, 9, 1, 2, 3, 4], 9)
```

```
Out[101]: [7, 8, 1, 2, 3, 4, 9]
```

```
In [102]: 1 move_to_end(["a", "a", "a", "b"], "a")
```

```
Out[102]: ['b', 'a', 'a', 'a']
```

## Program 107

Question1

Create a function that takes a string and returns a string in which each character is repeated once.

#### Examples

`double_char("String")` → "SSttrriinnngg"

`double_char("Hello World!")` → "HHeellllloo WWoorrrlidd!!"

`double char("1234! ")` → "11223344!! \_"

```
In [103]: 1 def double_char(input_str):
          2     doubled_str = ""
          3
          4     for char in input_str:
          5         doubled_str += char * 2
          6
          7     return doubled_str
```

```
In [104]: 1 double_char("String")
```

```
Out[104]: 'SSttrriinnngg'
```

```
In [105]: 1 double_char("Hello World!")
```

```
Out[105]: 'HHeellllloo WWoorrrlidd!!'
```

```
In [106]: 1 double_char("1234!_ ")
```

```
Out[106]: '11223344!!_ '
```

## Program 108

Create a function that reverses a boolean value and returns the string "boolean expected" if another variable type is given.

#### Examples

`reverse(True)` → False

`reverse(False)` → True

`reverse(0)` → "boolean expected"

`reverse(None)` → "boolean expected"

```
In [107]: 1 def reverse(value):
          2     if isinstance(value, bool):
          3         return not value
          4     else:
          5         return "boolean expected"
```

```
In [108]: 1 reverse(True)
```

```
Out[108]: False
```

```
In [109]: 1 reverse(False)
```

```
Out[109]: True
```

```
In [110]: 1 reverse(0)
```

```
Out[110]: 'boolean expected'
```

```
In [111]: 1 reverse(None)
```

```
Out[111]: 'boolean expected'
```

## Program 109

Create a function that returns the thickness (in meters) of a piece of paper after folding it n number of times. The paper starts off with a thickness of 0.5mm.

### Examples

num\_layers(1) → "0.001m"

- Paper folded once is 1mm (equal to 0.001m)

num\_layers(4) → "0.008m"

- Paper folded 4 times is 8mm (equal to 0.008m)

num\_layers(21) → "1048.576m"

- Paper folded 21 times is 1048576mm (equal to 1048.576m)

```
In [112]: 1 def num_layers(n):
2         initial_thickness_mm = 0.5 # Initial thickness in millimeters
3         final_thickness_mm = initial_thickness_mm * (2 ** n)
4         final_thickness_m = final_thickness_mm / 1000 # Convert millimeter
5         return f"{final_thickness_m:.3f}m"
```

```
In [113]: 1 num_layers(1)
```

```
Out[113]: '0.001m'
```

```
In [114]: 1 num_layers(4)
```

```
Out[114]: '0.008m'
```

```
In [115]: 1 num_layers(21)
```

```
Out[115]: '1048.576m'
```

## Program 110

Create a function that takes a single string as argument and returns an ordered list containing the indices of all capital letters in the string.

### Examples

`index_of_caps("eDaBiT") → [1, 3, 5]`

`index_of_caps("eQuINoX") → [1, 3, 4, 6]`

`index_of_caps("determine") → []`

`index_of_caps("STRIKE") → [0, 1, 2, 3, 4, 5]`

`index_of_caps("sUn") → [1]`

```
In [116]: 1 def index_of_caps(word):  
2         # Use list comprehension to find indices of capital letters  
3         return [i for i, char in enumerate(word) if char.isupper()]
```

```
In [117]: 1 index_of_caps("eDaBiT")
```

```
Out[117]: [1, 3, 5]
```

```
In [118]: 1 index_of_caps("eQuINoX")
```

```
Out[118]: [1, 3, 4, 6]
```

```
In [119]: 1 index_of_caps("determine")
```

```
Out[119]: []
```

```
In [120]: 1 index_of_caps("STRIKE")
```

```
Out[120]: [0, 1, 2, 3, 4, 5]
```

```
In [121]: 1 index_of_caps("sUn")
```

```
Out[121]: [1]
```

## Program 111

Using list comprehensions, create a function that finds all even numbers from 1 to the given number.

### Examples

`find_even_nums(8) → [2, 4, 6, 8]`

`find_even_nums(4) → [2, 4]`

`find_even_nums(2) → [2]`

```
In [123]: 1 def find_even_nums(num):  
2         # Use a list comprehension to generate even numbers from 1 to num  
3         return [x for x in range(1, num + 1) if x % 2 == 0]
```

```
In [124]: 1 find_even_nums(8)
```

```
Out[124]: [2, 4, 6, 8]
```

```
In [125]: 1 find_even_nums(4)
```

```
Out[125]: [2, 4]
```

```
In [126]: 1 find_even_nums(2)
```

```
Out[126]: [2]
```

## Program 112

Create a function that takes a list of strings and integers, and filters out the list so that it returns a list of integers only.

### Examples

`filter_list([1, 2, 3, "a", "b", 4]) → [1, 2, 3, 4]`

`filter_list(["A", 0, "Edabit", 1729, "Python", 1729]) → [0, 1729]`

`filter_list(["Nothing", "here"]) → []`

```
In [127]: 1 def filter_list(lst):  
2         # Use a list comprehension to filter out integers  
3         return [x for x in lst if isinstance(x, int)]
```

```
In [128]: 1 filter_list([1, 2, 3, "a", "b", 4])
```

```
Out[128]: [1, 2, 3, 4]
```

```
In [129]: 1 filter_list(["A", 0, "Edabit", 1729, "Python", 1729])
```

```
Out[129]: [0, 1729, 1729]
```

```
In [130]: 1 filter_list(["A", 0, "Edabit", 1729, "Python", 1729])
```

```
Out[130]: [0, 1729, 1729]
```

```
In [131]: 1 filter_list(["Nothing", "here"])
```

```
Out[131]: []
```

## Program 113

Given a list of numbers, create a function which returns the list but with each element's index in the list added to itself. This means you add 0 to the number at index 0, add 1 to the number at index 1, etc...

### Examples



**add\_indexes([0, 0, 0, 0, 0]) → [0, 1, 2, 3, 4]**

**add\_indexes([1, 2, 3, 4, 5]) → [1, 3, 5, 7, 9]**

**add\_indexes([5, 4, 3, 2, 1]) → [5, 5, 5, 5, 5]**

```
In [132]: 1 def add_indexes(lst):
          2     # Use list comprehension to add index to each element
          3     return [i + val for i, val in enumerate(lst)]
```

```
In [133]: 1 add_indexes([0, 0, 0, 0, 0])
```

Out[133]: [0, 1, 2, 3, 4]

```
In [134]: 1 add_indexes([1, 2, 3, 4, 5])
```

Out[134]: [1, 3, 5, 7, 9]

```
In [135]: 1 add_indexes([5, 4, 3, 2, 1])
```

Out[135]: [5, 5, 5, 5, 5]

## Program 114

Create a function that takes the height and radius of a cone as arguments and returns the volume of the cone rounded to the nearest hundredth. See the resources tab for the formula.

### Examples

**cone\_volume(3, 2) → 12.57**

**cone\_volume(15, 6) → 565.49**

**cone\_volume(18, 0) → 0**

```
In [136]: 1 import math
          2
          3 def cone_volume(height, radius):
          4     if radius == 0:
          5         return 0
          6     volume = (1/3) * math.pi * (radius**2) * height
          7     return round(volume, 2)
```

```
In [137]: 1 cone_volume(3, 2)
```

Out[137]: 12.57

```
In [138]: 1 cone_volume(15, 6)
```

Out[138]: 565.49

```
In [139]: 1 cone_volume(18, 0)
```

```
Out[139]: 0
```

## Program 115

This Triangular Number Sequence is generated from a pattern of dots that form a triangle. The first 5 numbers of the sequence, or dots, are:

1, 3, 6, 10, 15

This means that the first triangle has just one dot, the second one has three dots, the third one has 6 dots and so on.

Write a function that gives the number of dots with its corresponding triangle number of the sequence.

### Examples

triangle(1) → 1

triangle(6) → 21

triangle(215) → 23220

```
In [140]: 1 def triangle(n):  
2         if n < 1:  
3             return 0  
4         return n * (n + 1) // 2
```

```
In [141]: 1 triangle(1)
```

```
Out[141]: 1
```

```
In [142]: 1 triangle(6)
```

```
Out[142]: 21
```

```
In [143]: 1 triangle(215)
```

```
Out[143]: 23220
```

## Program 116

Create a function that takes a list of numbers between 1 and 10 (excluding one number) and returns the missing number.

Examples

`missing_num([1, 2, 3, 4, 6, 7, 8, 9, 10]) → 5`

`missing_num([7, 2, 3, 6, 5, 9, 1, 4, 8]) → 10`

`missing_num([10, 5, 1, 2, 4, 6, 8, 3, 9]) → 7`

```
In [144]: 1 def missing_num(lst):  
2         total_sum = sum(range(1, 11)) # Sum of numbers from 1 to 10  
3         given_sum = sum(lst) # Sum of the given list of numbers  
4         missing = total_sum - given_sum  
5         return missing
```

```
In [145]: 1 missing_num([1, 2, 3, 4, 6, 7, 8, 9, 10])
```

Out[145]: 5

```
In [146]: 1 missing_num([7, 2, 3, 6, 5, 9, 1, 4, 8])
```

Out[146]: 10

```
In [147]: 1 missing_num([10, 5, 1, 2, 4, 6, 8, 3, 9])
```

Out[147]: 7

## Program 117

Write a function that takes a list and a number as arguments. Add the number to the end of the list, then remove the first element of the list. The function should then return the updated list.

Examples

`next_in_line([5, 6, 7, 8, 9], 1) → [6, 7, 8, 9, 1]`

`next_in_line([7, 6, 3, 23, 17], 10) → [6, 3, 23, 17, 10]`

`next_in_line([1, 10, 20, 42 ], 6) → [10, 20, 42, 6]`

`next_in_line([], 6) → "No list has been selected"`

```
In [148]: 1 def next_in_line(lst, num):
          2     if lst:
          3         lst.pop(0) # Remove the first element
          4         lst.append(num) # Add the number to the end
          5         return lst
          6     else:
          7         return "No list has been selected"
```

```
In [149]: 1 next_in_line([5, 6, 7, 8, 9], 1)
```

```
Out[149]: [6, 7, 8, 9, 1]
```

```
In [150]: 1 next_in_line([7, 6, 3, 23, 17], 10)
```

```
Out[150]: [6, 3, 23, 17, 10]
```

```
In [151]: 1 next_in_line([1, 10, 20, 42 ], 6)
```

```
Out[151]: [10, 20, 42, 6]
```

```
In [152]: 1 next_in_line([], 6)
```

```
Out[152]: 'No list has been selected'
```

## Program 118

Create the function that takes a list of dictionaries and returns the sum of people's budgets.

### Examples

```
get_budgets([
    { 'name': 'John', 'age': 21, 'budget': 23000 },
    { 'name': 'Steve', 'age': 32, 'budget': 40000 },
    { 'name': 'Martin', 'age': 16, 'budget': 2700 }
]) → 65700

get_budgets([
    { 'name': 'John', 'age': 21, 'budget': 29000 },
    { 'name': 'Steve', 'age': 32, 'budget': 32000 },
    { 'name': 'Martin', 'age': 16, 'budget': 1600 }
]) → 62600
```

```
In [153]: 1 def get_budgets(lst):
2         total_budget = sum(person['budget'] for person in lst)
3         return total_budget
4
5 # Test cases
6 budgets1 = [
7     {'name': 'John', 'age': 21, 'budget': 23000},
8     {'name': 'Steve', 'age': 32, 'budget': 40000},
9     {'name': 'Martin', 'age': 16, 'budget': 2700}
10 ]
11
12 budgets2 = [
13     {'name': 'John', 'age': 21, 'budget': 29000},
14     {'name': 'Steve', 'age': 32, 'budget': 32000},
15     {'name': 'Martin', 'age': 16, 'budget': 1600}
16 ]
```

```
In [154]: 1 get_budgets(budgets1)
```

Out[154]: 65700

```
In [155]: 1 get_budgets(budgets2)
```

Out[155]: 62600

## Program 119

Create a function that takes a string and returns a string with its letters in alphabetical order.

### Examples

alphabet\_soup("hello") → "ehllo"

alphabet\_soup("edabit") → "abdeit"

alphabet\_soup("hacker") → "acehkr"

alphabet\_soup("geek") → "eegk"

alphabet\_soup("javascript") → "aacijprstv"

```
In [156]: 1 def alphabet_soup(txt):
2         return ''.join(sorted(txt))
```

```
In [157]: 1 alphabet_soup("hello")
```

Out[157]: 'ehllo'

```
In [158]: 1 alphabet_soup("edabit")
```

Out[158]: 'abdeit'

```
In [159]: 1 alphabet_soup("hacker")
```

```
Out[159]: 'acehkr'
```

```
In [160]: 1 alphabet_soup("geek")
```

```
Out[160]: 'eegk'
```

```
In [161]: 1 alphabet_soup("javascript")
```

```
Out[161]: 'aacijprstv'
```

## Program 120

Suppose that you invest \$10,000 for 10 years at an interest rate of 6% compounded monthly. What will be the value of your investment at the end of the 10 year period?

Create a function that accepts the principal  $p$ , the term in years  $t$ , the interest rate  $r$ , and the number of compounding periods per year  $n$ . The function returns the value at the end of term rounded to the nearest cent.

For the example:

`compound_interest(10000, 10, 0.06, 12)` → 18193.97

Note that the interest rate is given as a decimal and  $n=12$  because with monthly compounding there are 12 periods per year. Compounding can also be done annually, quarterly, weekly, or daily.

Examples

`compound_interest(100, 1, 0.05, 1)` → 105.0

`compound_interest(3500, 15, 0.1, 4)` → 15399.26

`compound_interest(100000, 20, 0.15, 365)` → 2007316.26

```
In [162]: 1 def compound_interest(p, t, r, n):
2         # Calculate the compound interest using the formula
3         a = p * (1 + (r / n)) ** (n * t)
4         # Round the result to the nearest cent
5         return round(a, 2)
```

```
In [163]: 1 compound_interest(10000, 10, 0.06, 12)
```

```
Out[163]: 18193.97
```

```
In [164]: 1 compound_interest(100, 1, 0.05, 1)
```

```
Out[164]: 105.0
```

```
In [165]: 1 compound_interest(3500, 15, 0.1, 4)
```

```
Out[165]: 15399.26
```

```
In [166]: 1 compound_interest(100000, 20, 0.15, 365)
```

```
Out[166]: 2007316.26
```

## Program 121

Write a function that takes a list of elements and returns only the integers.

Examples

`return_only_integer([9, 2, "space", "car", "lion", 16])` → `[9, 2, 16]`

`return_only_integer(["hello", 81, "basketball", 123, "fox"])` → `[81, 123]`

`return_only_integer([10, "121", 56, 20, "car", 3, "lion"])` → `[10, 56, 20, 3]`

`return_only_integer(["String", True, 3.3, 1])` → `[1]`

```
In [167]: 1 def return_only_integer(lst):
2         # Use list comprehension to filter out integers
3         return [x for x in lst if isinstance(x, int) and not isinstance(x,
```

```
In [168]: 1 return_only_integer([9, 2, "space", "car", "lion", 16])
```

```
Out[168]: [9, 2, 16]
```

```
In [169]: 1 return_only_integer(["hello", 81, "basketball", 123, "fox"])
```

```
Out[169]: [81, 123]
```

```
In [170]: 1 return_only_integer([10, "121", 56, 20, "car", 3, "lion"])
```

```
Out[170]: [10, 56, 20, 3]
```

```
In [171]: 1 return_only_integer(["String", True, 3.3, 1])
```

```
Out[171]: [1]
```

## Program 122

Create a function that takes three parameters where:

- x is the start of the range (inclusive).
- y is the end of the range (inclusive).
- n is the divisor to be checked against.

Return an ordered list with numbers in the range that are divisible by the third parameter n.

Return an empty list if there are no numbers that are divisible by n.

### Examples

`list_operation(1, 10, 3) → [3, 6, 9]`

`list_operation(7, 9, 2) → [8]`

`list_operation(15, 20, 7) → []`

```
In [172]: 1 def list_operation(x, y, n):
          2     # Use list comprehension to generate the list of numbers divisible
          3     return [num for num in range(x, y + 1) if num % n == 0]
```

```
In [173]: 1 list_operation(1, 10, 3)
```

```
Out[173]: [3, 6, 9]
```

```
In [174]: 1 list_operation(7, 9, 2)
```

```
Out[174]: [8]
```

```
In [175]: 1 list_operation(15, 20, 7)
```

```
Out[175]: []
```

## Program 123

Create a function that takes in two lists and returns True if the second list follows the first list by one element, and False otherwise. In other words, determine if the second list is the first list shifted to the right by 1.

### Examples

`simon_says([1, 2], [5, 1]) → True`

`simon_says([1, 2], [5, 5]) → False`

`simon_says([1, 2, 3, 4, 5], [0, 1, 2, 3, 4]) → True`

`simon_says([1, 2, 3, 4, 5], [5, 5, 1, 2, 3]) → False`

### Notes:

- Both input lists will be of the same length, and will have a minimum length of 2.
- The values of the 0-indexed element in the second list and the n-1th indexed element in the first list do not matter.

```
In [176]: 1 def simon_says(list1, list2):
          2     # Check if the second list is the first list shifted to the right by 1
          3     return list1[:-1] == list2[1:]
```



```
In [177]: 1 simon_says([1, 2], [5, 1])
```

```
Out[177]: True
```

```
In [178]: 1 simon_says([1, 2], [5, 5])
```

```
Out[178]: False
```

```
In [179]: 1 simon_says([1, 2, 3, 4, 5], [0, 1, 2, 3, 4])
```

```
Out[179]: True
```

```
In [180]: 1 simon_says([1, 2, 3, 4, 5], [5, 5, 1, 2, 3])
```

```
Out[180]: False
```

## Program 124

A group of friends have decided to start a secret society. The name will be the first letter of each of their names, sorted in alphabetical order. Create a function that takes in a list of names and returns the name of the secret society.

### Examples

`society_name(["Adam", "Sarah", "Malcolm"]) → "AMS"`

`society_name(["Harry", "Newt", "Luna", "Cho"]) → "CHLN"`

`society_name(["Phoebe", "Chandler", "Rachel", "Ross", "Monica", "Joey"])`

```
In [181]: 1 def society_name(names):  
2         # Extract the first letter of each name, sort them, and join into a string  
3         secret_name = ''.join(sorted([name[0] for name in names]))  
4         return secret_name
```

```
In [182]: 1 society_name(["Adam", "Sarah", "Malcolm"])
```

```
Out[182]: 'AMS'
```

```
In [183]: 1 society_name(["Harry", "Newt", "Luna", "Cho"])
```

```
Out[183]: 'CHLN'
```

```
In [184]: 1 society_name(["Phoebe", "Chandler", "Rachel", "Ross", "Monica", "Joey"])
```

```
Out[184]: 'CJMPRR'
```

## Program 125

An isogram is a word that has no duplicate letters. Create a function that takes a string and returns either True or False depending on whether or not it's an "isogram".

### Examples

**is\_isogram("Algorism") → True**

**is\_isogram("PasSword") → False**

**- Not case sensitive.**

**is\_isogram("Consecutive") → False**

### Notes

**Ignore letter case (should not be case sensitive).**

**All test cases contain valid one word strings.**

```
In [185]: 1 def is_isogram(word):
          2
          3     word = word.lower()
          4
          5     # Create a set to store unique letters in the word
          6     unique_letters = set()
          7
          8
          9     for letter in word:
         10         # If the letter is already in the set, it's not an isogram
         11         if letter in unique_letters:
         12             return False
         13         # Otherwise, add it to the set
         14         unique_letters.add(letter)
         15
         16     return True
```

```
In [186]: 1 is_isogram("Algorism")
```

Out[186]: True

```
In [187]: 1 is_isogram("PasSword")
```

Out[187]: False

```
In [188]: 1 is_isogram("Consecutive")
```

Out[188]: False

## Program 126

Create a function that takes a string and returns True or False, depending on whether the characters are in order or not.

### Examples

**is\_in\_order("abc") → True**

**is\_in\_order("edabit") → False**

**is\_in\_order("123") → True**

**is\_in\_order("xyzz") → True**

## Notes

You don't have to handle empty strings.

```
In [189]: 1 def is_in_order(s):  
          2  
          3     return s == ''.join(sorted(s))
```

```
In [190]: 1 is_in_order("abc")
```

Out[190]: True

```
In [191]: 1 is_in_order("edabit")
```

Out[191]: False

```
In [192]: 1 is_in_order("123")
```

Out[192]: True

```
In [193]: 1 is_in_order("xyzz")
```

Out[193]: True

## Program 127

Create a function that takes a number as an argument and returns True or False depending on whether the number is symmetrical or not. A number is symmetrical when it is the same as its reverse.

### Examples

is\_symmetrical(7227) → True

is\_symmetrical(12567) → False

is\_symmetrical(44444444) → True

is\_symmetrical(9939) → False

is\_symmetrical(1112111) → True

```
In [194]: 1 def is_symmetrical(num):  
          2     # Convert the number to a string  
          3     num_str = str(num)  
          4  
          5     # Check if the string is equal to its reverse  
          6     return num_str == num_str[::-1]
```

```
In [195]: 1 is_symmetrical(7227)
```

Out[195]: True

```
In [196]: 1 is_symmetrical(12567)
```

```
Out[196]: False
```

```
In [197]: 1 is_symmetrical(44444444)
```

```
Out[197]: True
```

```
In [199]: 1 is_symmetrical(44444444)
```

```
Out[199]: True
```

```
In [200]: 1 is_symmetrical(1112111)
```

```
Out[200]: True
```

## Program 128

Given a string of numbers separated by a comma and space, return the product of the numbers.

### Examples

`multiply_nums("2, 3") → 6`

`multiply_nums("1, 2, 3, 4") → 24`

`multiply_nums("54, 75, 453, 0") → 0`

`multiply_nums("10, -2") → -20`

```
In [201]: 1 def multiply_nums(nums_str):
2         # Split the input string by comma and space, then convert to integers
3         nums = [int(num) for num in nums_str.split(", ")]
4
5         # Initialize the result with 1
6         result = 1
7
8         # Multiply all the numbers together
9         for num in nums:
10            result *= num
11
12        return result
```

```
In [202]: 1 multiply_nums("2, 3")
```

```
Out[202]: 6
```

```
In [203]: 1 multiply_nums("1, 2, 3, 4")
```

```
Out[203]: 24
```

```
In [204]: 1 multiply_nums("54, 75, 453, 0")
```

```
Out[204]: 0
```

```
In [205]: 1 multiply_nums("10, -2")
```

```
Out[205]: -20
```

## Program 129

Create a function that squares every digit of a number.

### Examples

`square_digits(9119)` → 811181

`square_digits(2483)` → 416649

`square_digits(3212)` → 9414

### Notes

The function receives an integer and must return an integer.

```
In [206]: 1 def square_digits(n):
2         # Convert the number to a string to iterate through its digits
3         num_str = str(n)
4
5         # Initialize an empty string to store the squared digits
6         result_str = ""
7
8         # Iterate through the digits
9         for digit in num_str:
10            # Square the digit and convert it back to an integer
11            squared_digit = int(digit) ** 2
12
13            # Append the squared digit to the result string
14            result_str += str(squared_digit)
15
16
17         return int(result_str)
```

```
In [207]: 1 square_digits(9119)
```

```
Out[207]: 811181
```

```
In [208]: 1 square_digits(2483)
```

```
Out[208]: 416649
```

```
In [209]: 1 square_digits(3212)
```

```
Out[209]: 9414
```

## Program 130

Create a function that sorts a list and removes all duplicate items from it.

### Examples

**setify([1, 3, 3, 5, 5]) → [1, 3, 5]**

**setify([4, 4, 4, 4]) → [4]**

**setify([5, 7, 8, 9, 10, 15]) → [5, 7, 8, 9, 10, 15]**

**setify([3, 3, 3, 2, 1]) → [1, 2, 3]**

```
In [210]: 1 def setify(lst):
          2
          3     unique_set = set(sorted(lst))
          4
          5     # Convert the set back to a list and return it
          6     return list(unique_set)
```

```
In [211]: 1 setify([1, 3, 3, 5, 5])
```

```
Out[211]: [1, 3, 5]
```

```
In [212]: 1 setify([4, 4, 4, 4])
```

```
Out[212]: [4]
```

```
In [213]: 1 setify([5, 7, 8, 9, 10, 15])
```

```
Out[213]: [5, 7, 8, 9, 10, 15]
```

```
In [214]: 1 setify([3, 3, 3, 2, 1])
```

```
Out[214]: [1, 2, 3]
```

## Program 131

Create a function that returns the mean of all digits.

### Examples

mean(42) → 3

mean(12345) → 3

mean(666) → 6

### Notes

- The mean of all digits is the sum of digits / how many digits there are (e.g. mean of digits in 512 is  $(5+1+2)/3$ (number of digits) =  $8/3=2$ ).
- The mean will always be an integer.

```
In [215]: 1 def mean(n):
           2     # Convert the number to a string to iterate through its digits
           3     n_str = str(n)
           4
           5     # Calculate the sum of digits
           6     digit_sum = sum(int(digit) for digit in n_str)
           7
           8     # Calculate the mean by dividing the sum by the number of digits
           9     digit_count = len(n_str)
          10     digit_mean = digit_sum / digit_count
          11
          12     return int(digit_mean)
```

```
In [216]: 1 mean(42)
```

Out[216]: 3

```
In [217]: 1 mean(12345)
```

Out[217]: 3

```
In [218]: 1 mean(666)
```

Out[218]: 6

## Program 132

Create a function that takes an integer and returns a list from 1 to the given number, where:

1. If the number can be divided evenly by 4, amplify it by 10 (i.e. return 10 times the number).
2. If the number cannot be divided evenly by 4, simply return the number.

### Examples

**amplify(4) → [1, 2, 3, 40]**

**amplify(3) → [1, 2, 3]**

**amplify(25) → [1, 2, 3, 40, 5, 6, 7, 80, 9, 10, 11, 120, 13, 14, 15, 160, 17, 18, 19, 200, 21, 22, 23, 240, 25]**

### Notes

- The given integer will always be equal to or greater than 1.
- Include the number (see example above).
- To perform this problem with its intended purpose, try doing it with list comprehensions. If that's too difficult, just solve the challenge any way you can.

```
In [219]: 1 def amplify(num):  
2         # Use a list comprehension to generate the list  
3         return [n * 10 if n % 4 == 0 else n for n in range(1, num + 1)]
```

```
In [220]: 1 amplify(4)
```

```
Out[220]: [1, 2, 3, 40]
```

```
In [221]: 1 amplify(3)
```

```
Out[221]: [1, 2, 3]
```

```
In [222]: 1 amplify(25)
```

```
Out[222]: [1,  
2,  
3,  
40,  
5,  
6,  
7,  
80,  
9,  
10,  
11,  
120,  
13,  
14,  
15,  
160,  
17,  
18,  
19,  
200,  
21,  
22,  
23,  
240,  
25]
```

## Program 133

**Create a function that takes a list of numbers and return the number that's unique.**



## Examples

`unique([3, 3, 3, 7, 3, 3]) → 7`

`unique([0, 0, 0.77, 0, 0]) → 0.77`

`unique([0, 1, 1, 1, 1, 1, 1, 1]) → 0`

## Notes

Test cases will always have exactly one unique number while all others are the same.

In [223]:

```

1 def unique(numbers):
2     # Use a dictionary to count occurrences of each number
3     count_dict = {}
4
5     # Count occurrences of each number in the list
6     for num in numbers:
7         if num in count_dict:
8             count_dict[num] += 1
9         else:
10            count_dict[num] = 1
11
12    # Find the unique number (occurs only once)
13    for num, count in count_dict.items():
14        if count == 1:
15            return num

```

In [224]:

```
1 unique([3, 3, 3, 7, 3, 3])
```

Out[224]: 7

In [225]:

```
1 unique([0, 0, 0.77, 0, 0])
```

Out[225]: 0.77

In [226]:

```
1 unique([0, 1, 1, 1, 1, 1, 1, 1])
```

Out[226]: 0

## Program 134

Your task is to create a Circle constructor that creates a circle with a radius provided by an argument. The circles constructed must have two getters `getArea()` ( $\pi r^2$ ) and `getPerimeter()` ( $2\pi r$ ) which give both respective areas and perimeter (circumference).

For help with this class, I have provided you with a Rectangle constructor which you can use as a base example.

## Examples

`circy = Circle(11)`

`circy.getArea()`

- Should return 380.132711084365

circy = Circle(4.44)

circy.getPerimeter()

- Should return 27.897342763877365

### Notes

Round results up to the nearest integer.

In [227]:

```
1 import math
2
3 class Circle:
4     def __init__(self, radius):
5         self.radius = radius
6
7     def getArea(self):
8         # Calculate and return the area of the circle
9         return round(math.pi * self.radius**2)
10
11    def getPerimeter(self):
12        # Calculate and return the perimeter (circumference) of the circle
13        return round(2 * math.pi * self.radius)
```

In [228]:

```
1 # Test cases
2 circy = Circle(11)
3 print(circy.getArea())
4 print(circy.getPerimeter())
```

380  
69

In [229]:

```
1 circy = Circle(4.44)
2 print(circy.getArea())
3 print(circy.getPerimeter())
```

62  
28

## Program 135

Create a function that takes a list of strings and return a list, sorted from shortest to longest.

### Examples

sort\_by\_length(["Google", "Apple", "Microsoft"]) → ["Apple", "Google", "Microsoft"]

sort\_by\_length(["Leonardo", "Michelangelo", "Raphael", "Donatello"]) → ["Raphael", "Leonardo", "Donatello", "Michelangelo"]

sort\_by\_length(["Turing", "Einstein", "Jung"]) → ["Jung", "Turing", "Einstein"]

### Notes

**All test cases contain lists with strings of different lengths, so you won't have to deal**

```
In [230]: 1 def sort_by_length(lst):  
2  
3         return sorted(lst, key=len)      # Using sorted() function with a cu
```

```
In [231]: 1 sort_by_length(["Google", "Apple", "Microsoft"])
```

```
Out[231]: ['Apple', 'Google', 'Microsoft']
```

```
In [232]: 1 sort_by_length(["Leonardo", "Michelangelo", "Raphael", "Donatello"])
```

```
Out[232]: ['Raphael', 'Leonardo', 'Donatello', 'Michelangelo']
```

```
In [233]: 1 sort_by_length(["Turing", "Einstein", "Jung"])
```

```
Out[233]: ['Jung', 'Turing', 'Einstein']
```

## Program 136

Create a function that validates whether three given integers form a Pythagorean triplet. The sum of the squares of the two smallest integers must equal the square of the largest number to be validated.

### Examples

`is_triplet(3, 4, 5) → True`

- $3^2 + 4^2 = 25$
- $5^2 = 25$

`is_triplet(13, 5, 12) → True`

- $5^2 + 12^2 = 169$
- $13^2 = 169$

`is_triplet(1, 2, 3) → False`

- $1^2 + 2^2 = 5$
- $3^2 = 9$

### Notes

**Numbers may not be given in a sorted order.**

```
In [234]: 1 def is_triplet(a, b, c):  
2         # Sort the numbers in ascending order  
3         sorted_numbers = sorted([a, b, c])  
4         # Check if the sum of squares of the two smaller numbers equals the  
5         return sorted_numbers[0] ** 2 + sorted_numbers[1] ** 2 == sorted_nu
```

```
In [235]: 1 is_triplet(3, 4, 5)
```

```
Out[235]: True
```

```
In [236]: 1 is_triplet(13, 5, 12)
```

```
Out[236]: True
```

```
In [237]: 1 is_triplet(1, 2, 3)
```

```
Out[237]: False
```

## Program 137

Create a function that takes three integer arguments (a, b, c) and returns the amount of integers which are of equal value.

### Examples

`equal(3, 4, 3) → 2`

`equal(1, 1, 1) → 3`

`equal(3, 4, 1) → 0`

### Notes

Your function must return 0, 2 or 3.

```
In [238]: 1 def equal(a, b, c):  
2         if a == b == c:  
3             return 3  
4         elif a == b or b == c or a == c:  
5             return 2  
6         else:  
7             return 0
```

```
In [239]: 1 equal(3, 4, 3)
```

```
Out[239]: 2
```

```
In [240]: 1 equal(1, 1, 1)
```

```
Out[240]: 3
```

```
In [241]: 1 equal(3, 4, 1)
```

```
Out[241]: 0
```

## Program 138

Write a function that converts a dictionary into a list of keys-values tuples.

### Examples

`dict_to_list({`

`"D": 1,`

"B": 2

"C": 3

} → [("B", 2), ("C", 3), ("D", 1)]

dict\_to\_list({

"likes": 2,

"dislikes": 3,

"followers": 10

} → [("dislikes", 3), ("followers", 10), ("likes", 2)]

### Notes

Return the elements in the list in alphabetical order.

```
In [242]: 1 def dict_to_list(input_dict):
2         # Sort the dictionary by keys in alphabetical order
3         sorted_dict = sorted(input_dict.items())
4
5         # Convert the sorted dictionary to a list of tuples
6         result = [(key, value) for key, value in sorted_dict]
7
8         return result
```

```
In [243]: 1 dict_to_list({
2         "D": 1,
3         "B": 2,
4         "C": 3
5     })
```

Out[243]: [('B', 2), ('C', 3), ('D', 1)]

```
In [244]: 1 dict_to_list({
2
3         "likes": 2,
4
5         "dislikes": 3,
6
7         "followers": 10
8
9     })
```

Out[244]: [('dislikes', 3), ('followers', 10), ('likes', 2)]

## Program 139

Write a function that creates a dictionary with each (key, value) pair being the (lower case, upper case) versions of a letter, respectively.

### Examples

mapping(["p", "s"]) → { "p": "P", "s": "S" }

**mapping(["a", "b", "c"]) → { "a": "A", "b": "B", "c": "C" }**

**mapping(["a", "v", "y", "z"]) → { "a": "A", "v": "V", "y": "Y", "z": "Z" }**

### Notes

**All of the letters in the input list will always be lowercase.**

```
In [245]: 1 def mapping(letters):
          2     result = {}
          3     for letter in letters:
          4         result[letter] = letter.upper()
          5     return result
```

```
In [246]: 1 mapping(["p", "s"])
```

```
Out[246]: {'p': 'P', 's': 'S'}
```

```
In [247]: 1 mapping(["a", "b", "c"])
```

```
Out[247]: {'a': 'A', 'b': 'B', 'c': 'C'}
```

```
In [248]: 1 mapping(["a", "v", "y", "z"])
```

```
Out[248]: {'a': 'A', 'v': 'V', 'y': 'Y', 'z': 'Z'}
```

## Program 140

**Write a function, that replaces all vowels in a string with a specified vowel.**

### Examples

**vow\_replace("apples and bananas", "u") → "upplus und bununus"**

**vow\_replace("cheese casserole", "o") → "chooso cossorolo"**

**vow\_replace("stuffed jalapeno poppers", "e") → "steffed jelepene peppers"**

### Notes

**All words will be lowercase. Y is not considered a vowel.**

```
In [249]: 1 def vow_replace(string, vowel):
          2     vowels = "aeiou"
          3     result = ""
          4
          5     for char in string:
          6         if char in vowels:
          7             result += vowel
          8         else:
          9             result += char
         10
         11     return result
```

```
In [250]: 1 vow_replace("apples and bananas", "u")
```

```
Out[250]: 'upplus und bununus'
```

```
In [251]: 1 vow_replace("cheese casserole", "o")
```

```
Out[251]: 'chooso cossorolo'
```

```
In [252]: 1 vow_replace("stuffed jalapeno poppers", "e")
```

```
Out[252]: 'steffed jelepene peppers'
```

## Program 141

Create a function that takes a string as input and capitalizes a letter if its ASCII code is even and returns its lower case version if its ASCII code is odd.

### Examples

`ascii_capitalize("to be or not to be!")` → "To Be oR NoT To Be!"

`ascii_capitalize("THE LITTLE MERMAID")` → "The LiTTLe meRmaiD"

`ascii_capitalize("Oh what a beautiful morning.")` → "oH wHaT a BeauTiFuL moRniNg."

```
In [253]: 1 def ascii_capitalize(input_str):
2         result = ""
3
4         for char in input_str:
5             if ord(char) % 2 == 0:
6                 result += char.upper()
7             else:
8                 result += char.lower()
9
10        return result
```

```
In [254]: 1 ascii_capitalize("to be or not to be!")
```

```
Out[254]: 'To Be oR NoT To Be!'
```

```
In [255]: 1 ascii_capitalize("THE LITTLE MERMAID")
```

```
Out[255]: 'The LiTTLe meRmaiD'
```

```
In [256]: 1 ascii_capitalize("Oh what a beautiful morning.")
```

```
Out[256]: 'oH wHaT a BeauTiFuL moRniNg.'
```

